Remote Test Jig

User Manual — XIAO nRF52840 bench test suite for BLE and IR TV remotes

Generated 2026-10-29 from doc/*.md

Contents

Remote Test Jig — User Manual
- Start here
- The one decision that matters
- Figures in this manual

1 · Overview
- What the jig does
- System overview
- Two firmware images — which one, when
- Remotes tested

2 · Hardware
- Parts
- IR wiring
- Battery and charging
- Enclosure

3 · Installation
- 3.1 Python environment (always needed)
- 3.2 Serial port permission (Linux, once)
- 3.3 Firmware toolchain (only to rebuild the firmware)
- 3.4 Wireshark (only for passive capture)
- 3.5 Phone access

4 · Firmware and flashing
- Which firmware?
- How flashing works
- Option A — from the web app
- Option B — with the helper
- Option C — by hand
- Rules
- Checking what runs

5 · Quick start
- 1 · Plug in the XIAO
- 2 · Start the bridge
- 3 · Open the app
- 4 · Check the banner
- 5 · Pair the remote
- 6 · Press keys
- 7 · Run the test and export
- Next

6 · Web app tour
- Banner
- Connect
- Keys
- Voice
- IR
- Test
- On a phone

7 · Pairing and remotes
- What happens over the air
- Pairing a remote
- How the firmware handles links
- Remotes differ on reconnect
- LG Magic Remote
- Fire TV voice
- Bonds and the session lock
- Addresses

8 · Testing and reports
- Before you start
- Running
- Settings
- Exporting the report
- Tips for production use

9 · Passive capture (Nordic sniffer + Wireshark)
- When to use it — and its limit
- Flash the sniffer image
- Install the plugin
- Capture
- Terminal alternative: `ble_probe.py`

10 · Troubleshooting
- Banner and connection
- Pairing and links
- Keys, voice, battery
- IR
- Flashing
- Collecting information for a bug report

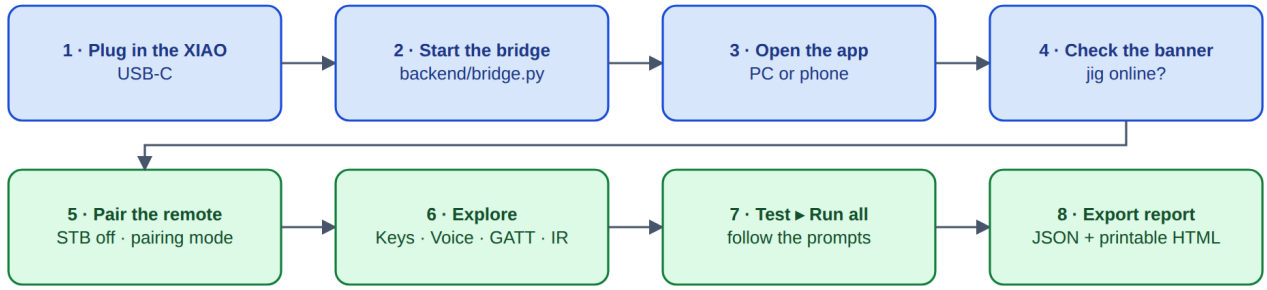
11 · Reference
- Bridge command line
- HTTP endpoints (bridge)
- Serial protocol (xiao-test-tool)
- Reason and error codes
- Pins
- Files
- Glossary

Remote Test Jig — User Manual

A bench test suite for TV remotes (BLE voice remotes and IR remotes), built on one **Seeed XIAO nRF52840**, a small Python bridge on a PC, and a web app for a PC or phone.

A test session at a glance

Typical bench routine with the xiao-test-tool firmware (Ch. 5 Quick start).



Start here

If you want to…	Read
understand what the system is and **which firmware to use for which job**	1 · Overview
wire the IR unit, fit a battery, print the case	2 · Hardware
install the software on a PC	3 · Installation
flash or switch firmware	4 · Firmware and flashing
get from zero to decoded key presses in 10 minutes	5 · Quick start
learn every screen of the web app	6 · Web app tour
pair a remote; understand reconnects and “silent” links	7 · Pairing and remotes
run the pass/fail test and export a report	8 · Testing and reports
watch a real remote ⇄ STB link in Wireshark	9 · Passive capture
fix a problem	10 · Troubleshooting
look up the serial protocol, CLI options, pins and files	11 · Reference

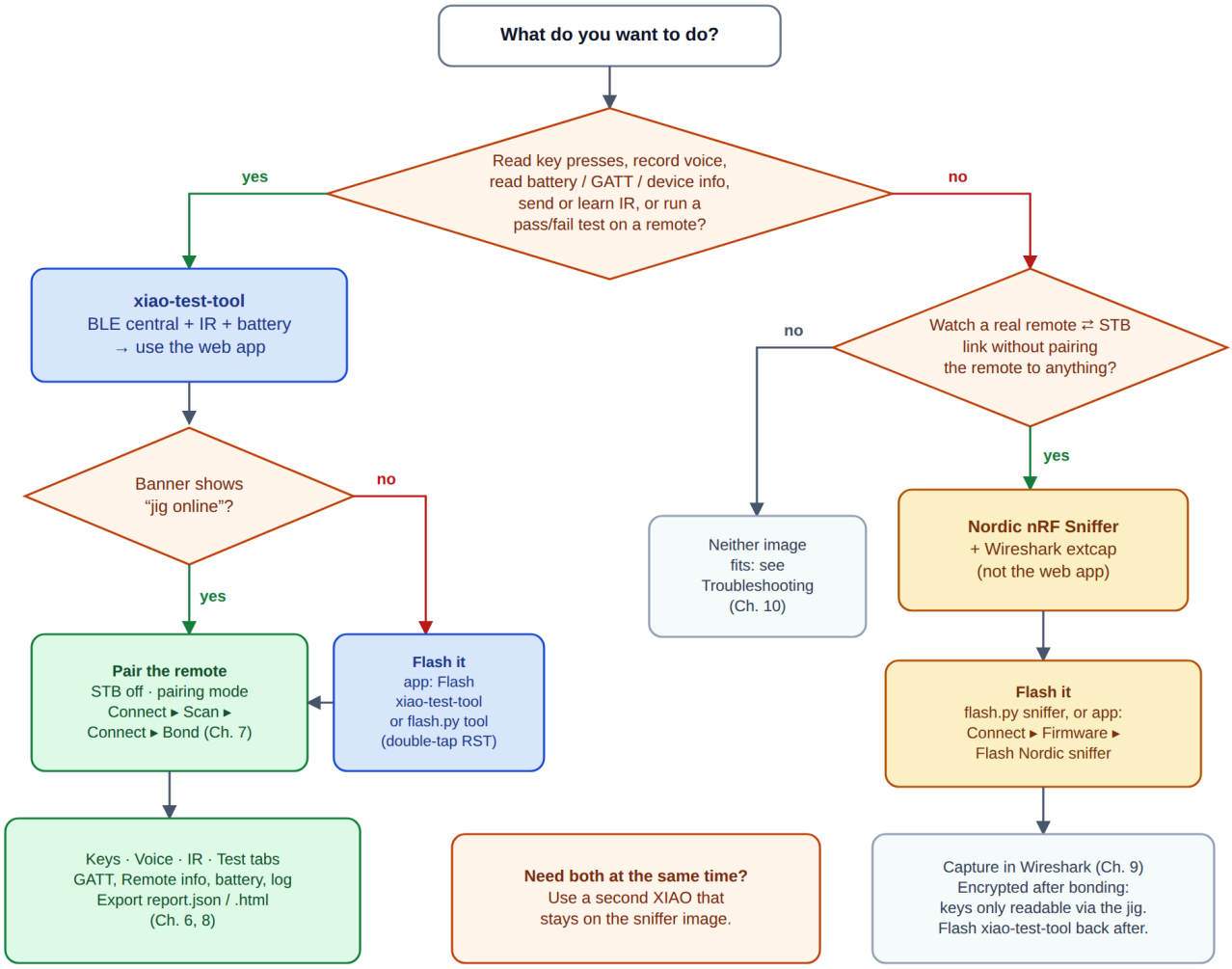
The one decision that matters

The XIAO runs **one of two firmware images**:

Firmware	Use it to…	Front end
xiao-test-tool (built from this repo)	pair with a remote and read **keys, voice, battery, GATT/device info**; send/learn **IR**; run the **pass/fail test**	this web app
Nordic nRF Sniffer 4.1	**passively watch** a real remote ⇄ STB link without touching it	Wireshark

Which firmware for which job?

Only one image runs at a time. Switching takes ~20 s (Ch. 4). For both roles at once, use a second XIAO.



Figures in this manual

Figure	Shows
architecture	how the remote, XIAO, PC bridge and browsers connect — for both firmware images
firmware-decision	which firmware for which job
flashing-flow	switching images through the UF2 bootloader
pairing-flow	the firmware's connect / pair / silent-link logic
remote_lifecycle_sequence	what goes over the air between remote and jig
ir_connection	wiring the M5Stack Unit IR
workflow	a test session at a glance

Every figure exists as `.svg` (scalable, for screens) and `.png` (2×, for documents and printing). Screenshots are in `images/`. How to regenerate both: `src/README.md`.

PDF: the whole manual as one printable file — `python3 doc/src/make_pdf.py` → `Remote-Test-Jig-User-Manual.pdf`.

The top-level `README.md` is the condensed operator guide; this manual is the full version.

1 · Overview

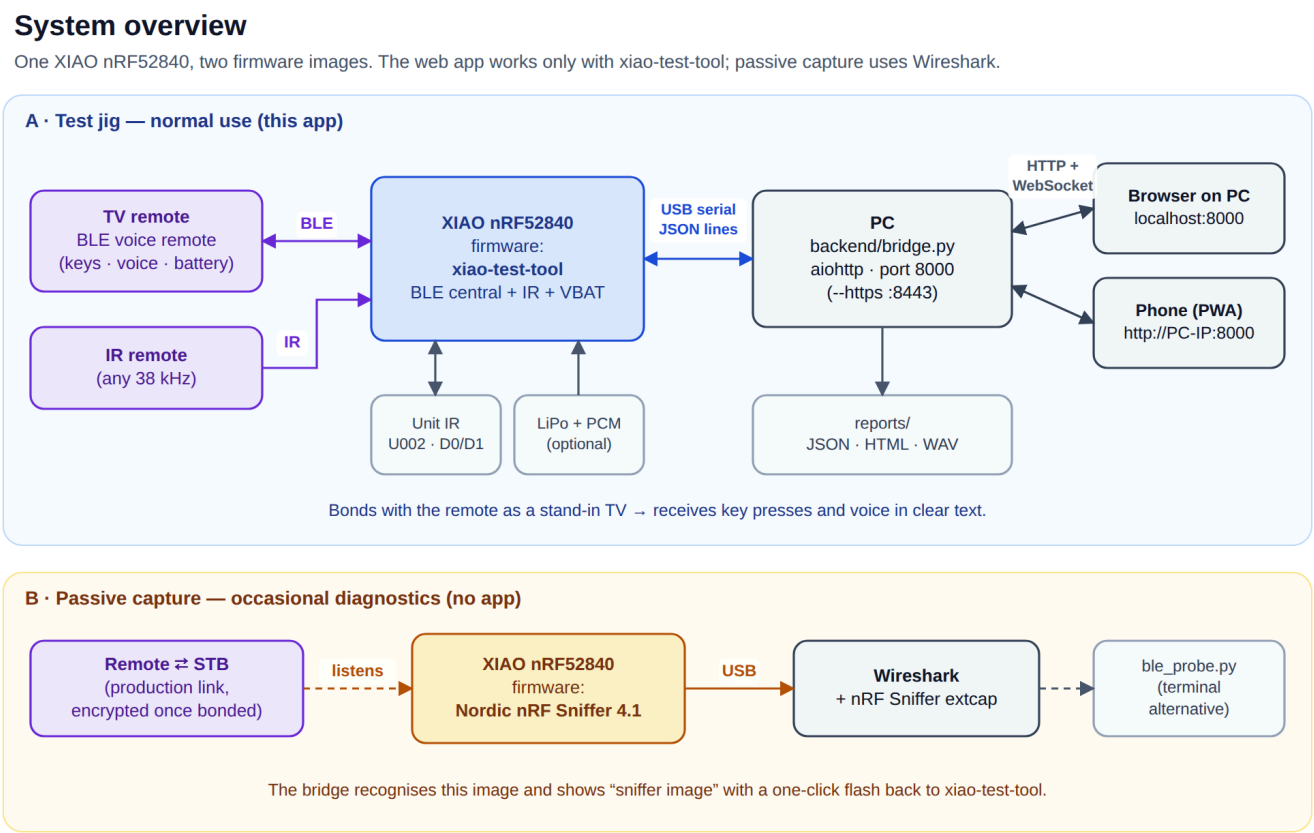
What the jig does

The **Remote Test Jig** takes the place of the TV or set-top box (STB) on a bench. With it you can:

- pair with a BLE TV remote and see every **key press decoded** (`VOLUME_UP`, `HOME`, `DIGIT_5` …) with its latency;
- read everything the remote reports about itself: **GATT table, Device Information** (manufacturer, model, serial, firmware), **PnP ID**, **HID report map**, **battery level**;
- capture the remote's **voice** (Android TV Voice service) and save it as WAV;
- **receive, learn and send IR** codes through an M5Stack Unit IR;
- watch the jig's own **battery**;
- run a scripted **pass/fail test** and export `report.json` + a printable `report.html`;
- keep a timestamped **log** of every GATT read and notification, and copy or download it.

It is driven from a browser on the PC or a phone on the same Wi-Fi (installable as an app).

System overview



Part	What it is
XIAO nRF52840	Seeed's thumb-sized board with a Nordic nRF52840 (Bluetooth LE 5). The only radio in the system.
Firmware	One of two images, chosen by flashing — see below.

Part	What it is
Bridge (`backend/bridge.py`)	Python program on the PC. Talks to the XIAO over USB serial, serves the web app and a WebSocket on port 8000.
Web app (`webapp/`)	Runs in any browser. Tabs: Connect, Keys, Voice, IR, Test.
Unit IR (optional)	M5Stack U002: IR LED + 38 kHz receiver in one unit, wired to D0/D1.
LiPo (optional)	Makes the jig portable; charged by the XIAO over USB-C.

Two firmware images — which one, when

The nRF52840 can only run one firmware at a time, and the two jobs need different firmware.

xiao-test-tool — the test jig (normal use)

Built from `firmware/xiao_test_tool/`. The XIAO acts as a **BLE central (host)**: it **pairs with the remote itself**, exactly as a TV would. Because it holds the encryption keys, it receives the remote's key presses and voice **in clear text**. It also drives the IR unit and measures its battery.

Use it for everything the web app does: keys, voice, GATT/device information, battery, IR, the test runner and reports.

Consequence: pairing bonds the remote to the **jig**. Many remotes remember only one host, so you may have to pair the remote with its STB again after testing.

Nordic nRF Sniffer — passive capture (diagnostics)

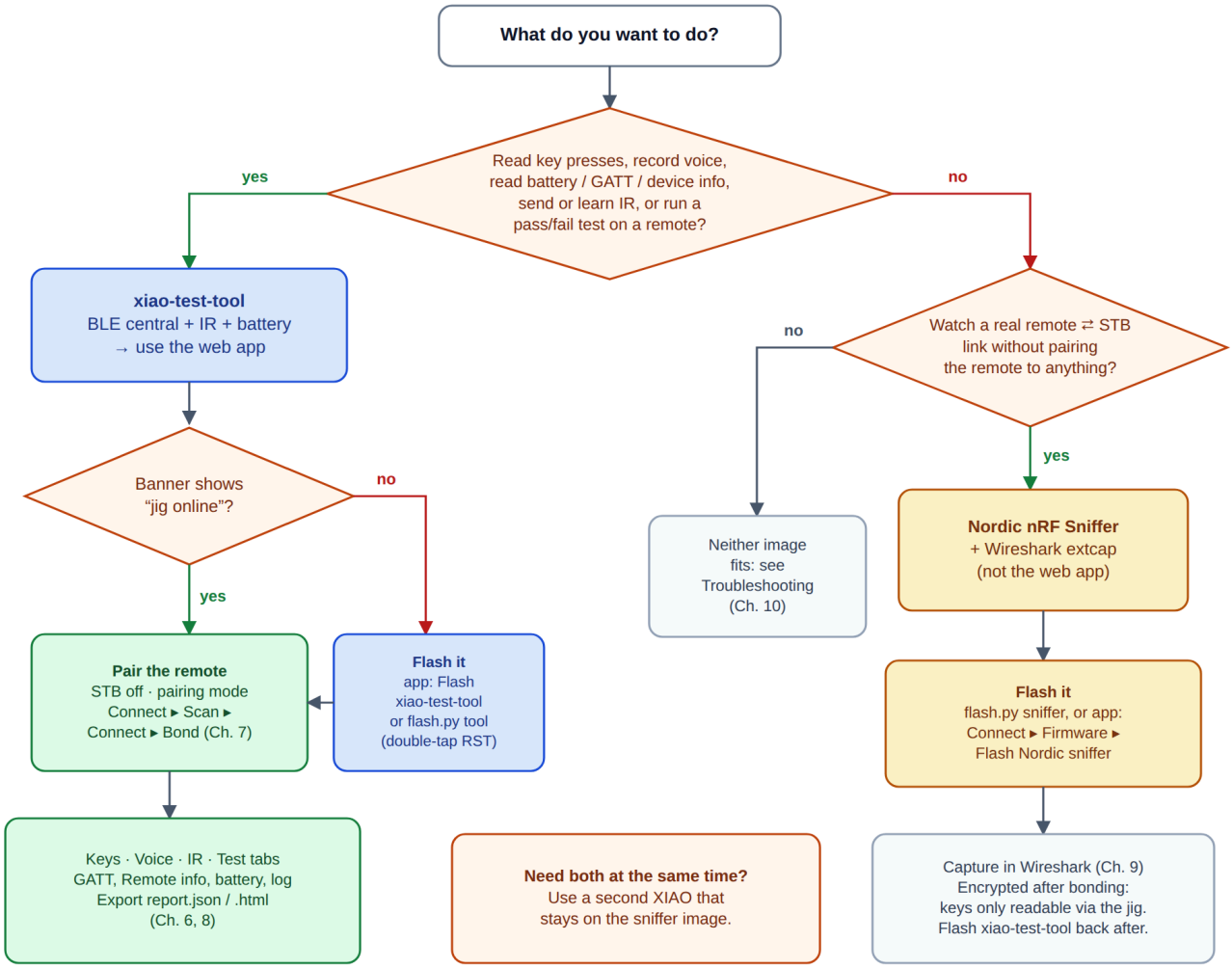
Nordic's closed `sniffer_nrf52840dongle_4.1.0.uf2`. The XIAO only **listens** to a link between a remote and its real STB and hands the packets to **Wireshark**. The web app is not used.

Limit: once a remote is bonded, its link is **encrypted**. The sniffer shows connection set-up, timing and packet sizes, but not key presses — unless the capture includes the pairing itself and the remote uses legacy pairing (see Passive capture). To read keys, use the jig.

Decision chart

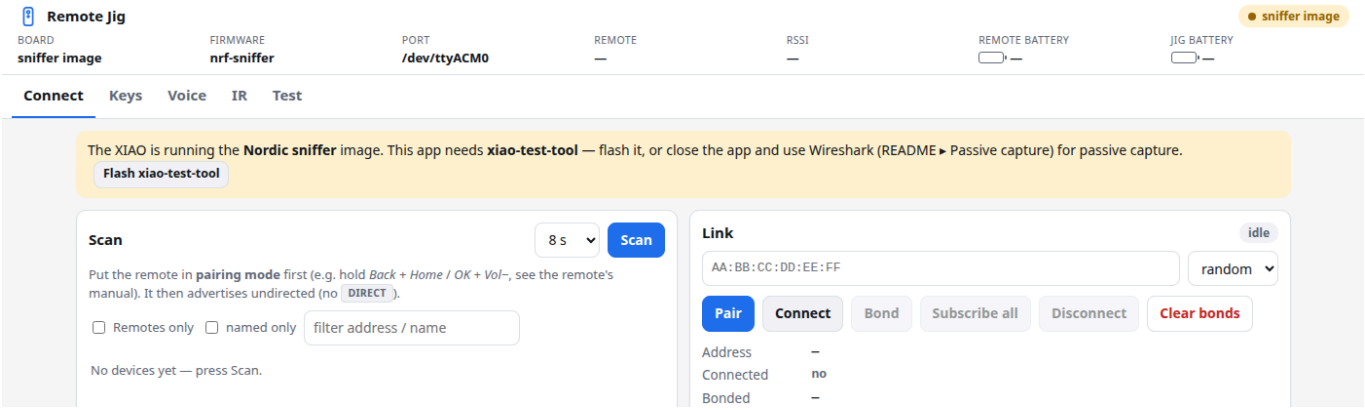
Which firmware for which job?

Only one image runs at a time. Switching takes ~20 s (Ch. 4). For both roles at once, use a second XIAO.



Situation	Firmware	Where to go
Check that every button of a remote works	xiao-test-tool	Quick start, Testing
Record and listen to the voice / mic path	xiao-test-tool	Web app ▸ Voice
Read model, serial, firmware version, battery of a remote	xiao-test-tool	Web app ▸ Connect
Learn or replay IR codes, IR loopback test	xiao-test-tool (+ Unit IR)	Web app ▸ IR
Production test with a printed report	xiao-test-tool	Testing and reports
See how a remote talks to **its real STB** (timing, reconnects, PHY)	Nordic sniffer	Passive capture
Debug why a remote won't reconnect to the STB	Nordic sniffer	Passive capture
Both at the same time	two XIAOs, one per image	—

You can always see which image is running: the banner at the top of the web app shows **jig online** (xiao-test-tool) or **sniffer image** (Nordic), and offers a one-click flash back.



Remotes tested

Remote	Address	Keys	Voice	Notes
Android TV voice remote	60:74:B1:80:AF:1E	yes	yes (ATV Voice v1.0, 16 kHz)	only sends on a freshly paired link — pairing mode per session (Ch. 7)
Fire TV remote (“AR”)	30:4D:1F:13:82:8B	yes	not yet (Amazon’s own voice protocol)	reconnects normally; press any key to wake

2 · Hardware

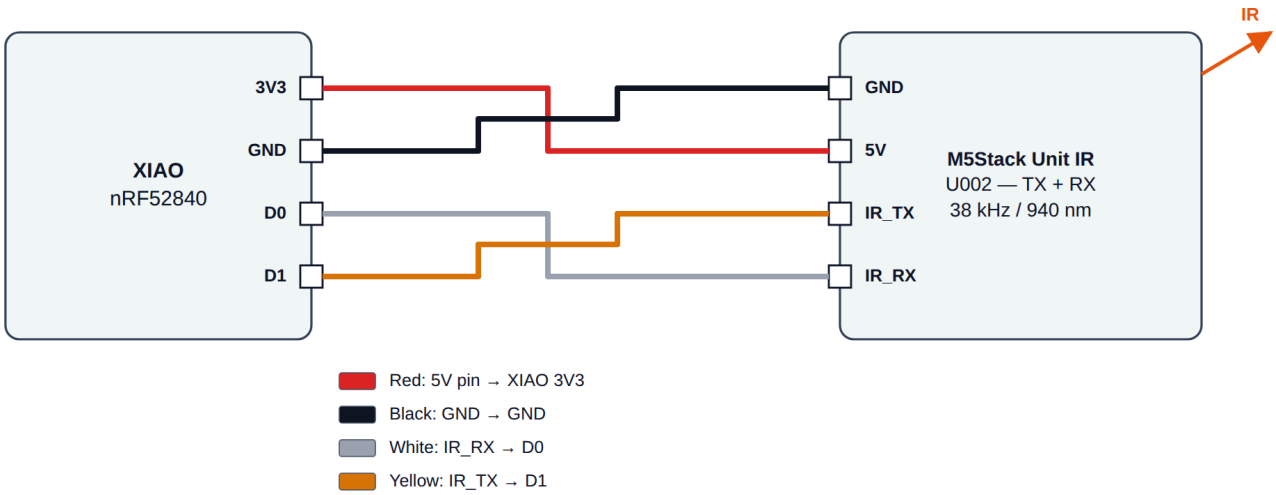
Parts

Part	Needed for	Notes
Seeed **XIAO nRF52840** (plain or Sense)	everything	the drive in bootloader mode is named `XIAO-BOOT` or `XIAO-SENSE`
USB-C **data** cable	everything	charge-only cables don't enumerate
PC with Linux (Ubuntu tested)	everything	macOS/Windows run the bridge; the flash helper is Linux/macOS-oriented
M5Stack **Unit IR (U002)**	IR tab, IR loopback test	TX + RX in one unit, 4-wire HY2.0 lead
1-cell **LiPo 41 × 10 × 6 mm with protection PCM**	battery operation	never a bare cell — see Battery safety
Printed enclosure	optional	`hardware/bt_case_base.stl` + `bt_case_lid.stl`
Soft pads (TPU or 1 mm silicone bumpers)	with the enclosure	carry the board above the cell

IR wiring

IR wiring — XIAO nRF52840 + M5Stack Unit IR (U002)

One 4-wire HY2.0 cable. TX and RX in one unit (38 kHz, 940 nm). Firmware makes the 38 kHz on D1 and decodes on D0.



Power the unit from 3V3, not 5V
The nRF52840 is not 5V-tolerant and IR_RX follows the supply. 3V3 is also the only rail on battery.
Max range (USB only): 5V on red + a divider on IR_RX (10 kΩ series, 20 kΩ to GND).

Pins and mounting
D0 = P0.02 ← IR_RX (demodulated, active low)
D1 = P0.03 → IR_TX (38 kHz PWM carrier, ~33 % duty)
No extra parts: the unit drives its own LED. Solder the 4 wires to the pads; the unit mounts outside the case.

Unit IR wire	XIAO pad	nRF52840 pin	Function
Black (GND)	GND	—	ground
Red (5V)	**3V3**	—	power — **not 5V**

Unit IR wire	XIAO pad	nRF52840 pin	Function
Yellow (IR_TX)	D1	P0.03	38 kHz carrier from the firmware (`HwPWM3` , ~33 % duty)
White (IR_RX)	D0	P0.02	demodulated receiver output, active low, internal pull-up

- **Power from 3V3.** The nRF52840 is not 5V-tolerant and the unit's IR_RX output follows its supply. 3V3 is also the only rail available on battery.
- **More range (USB power only):** red to 5V, and a divider on IR_RX into D0 (10 kΩ in series, 20 kΩ to GND). TX stays direct.
- No extra parts: the unit has its own LED driver. Cut the HY2.0 lead and solder the four wires to the XIAO pads. The unit mounts **outside** the case, facing the remote (it needs line of sight).

Battery and charging

The jig runs from USB alone. For portable use, solder a protected 1-cell LiPo to the `BAT+` / `BAT−` pads under the XIAO. The on-board **BQ25101** charges it over USB-C (about 50 mA, set by an on-board resistor — fine for a small pouch; don't raise it unless the cell's datasheet allows).

The firmware reads the cell on P0.31 through the on-board 1 MΩ / 510 kΩ divider (enabled by P0.14, which the firmware holds LOW permanently — releasing it would push P0.31 above its safe range).

Reading	What the jig does
≥ 3.4 V	normal
< 3.4 V	`battery` event `low: true` ; banner marks **LOW**, one-time warning
< 3.3 V	`cut: true` ; **IR transmit refused** (heaviest load) until the cell is back above 3.4 V
< 2.5 V	treated as "no cell fitted" (USB only) — no cut
~4.2 V with no cell	normal: the VBAT pin sees the charger output

Battery safety

The BQ25101 is a **charger, not a protection circuit**:

Failure	Protected by	Automatic?
Overcharge (> 4.2 V)	BQ25101 charger	yes
Over-discharge	the **cell's own PCM**	only if the cell has one
Over-current / short	the **cell's own PCM**	only if the cell has one
Early low-battery warning	firmware + app (above)	yes — a soft limit, never the safety net

Always use a cell **with** a protection PCM. The real fire risks are mechanical and manual:

1. **Reverse polarity** on `BAT+ / BAT−` — check three times before first power-up.
2. **Crushing or puncturing the pouch** — the case keeps the board on soft stand-offs above the cell and has a hard spacer before the USB-C slot; deburr the print.
3. **Heat on the cell body** while soldering — solder fast to the PCM leads.
4. **A swollen or damaged cell** — never charge it; dispose of it safely.

Enclosure

File	What
`hardware/bt_case_base.stl`, `bt_case_lid.stl`	the provided case (≈ 46 × 23 × 14 mm): pouch channel with rails, corner stand-offs ~0.3 mm above the pouch, USB-C slot, cable slot for the IR lead, flush friction lid
`hardware/bt_case.scad`	parametric alternative → `bt_case_param_base.stl`, `bt_case_param_lid.stl` (notched over USB-C), `bt_case_param_pads.stl` (four 3 × 3 × 1 mm pads, print in TPU)

Re-render after editing the `.scad`:

```
openscad -D 'part="base"' -o hardware/bt_case_param_base.stl hardware/bt_case.scad   # likewise
        lid, pads
```

Check the fit against your actual board and cell before printing.

3 · Installation

Everything installs **inside the project folder** (`.venv/` , `.toolchain/`); nothing goes into your home directory's Arduino settings.

3.1 Python environment (always needed)

```
cd BT-Sniffer
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

Package	Used by
`aiohttp`	the bridge: web server + WebSocket
`pyserial`	USB serial to the XIAO (also `ble_probe.py` and the Wireshark plugin)
`intelhex`	`firmware/make_uf2.py` — builds the full UF2 with the SoftDevice
`adafruit-nrfutil`	building the firmware with arduino-cli

Nordic's **SnifferAPI** is not on PyPI; it is vendored in `extcap/SnifferAPI/` and found automatically.

Check:

```
.venv/bin/python backend/bridge.py --simulate
# open http://localhost:8000 — a simulated jig and remote, no hardware needed. Ctrl+C to stop.
        {#03-installation--open-httplocalhost8000--a-simulated-jig-and-remote-no-hardware-
        needed-ctrlc-to-stop}
```

3.2 Serial port permission (Linux, once)

```
sudo usermod -aG dialout $USER      # then log out and back in
```

Without it the bridge reports *Permission denied* on `/dev/ttyACM0` .

3.3 Firmware toolchain (only to rebuild the firmware)

A ready UF2 is produced in `firmware/xiao_test_tool/build/` . To build it yourself:

```
mkdir -p .toolchain/bin && cd .toolchain
curl -fsSL https://github.com/arduino/arduino-cli/releases/download/v1.1.1/arduino-
        cli_1.1.1_Linux_64bit.tar.gz \
  | tar xz -C bin arduino-cli
cat > arduino-cli.yaml <<EOF
board_manager:
  additional_urls: [https://files.seeedstudio.com/arduino/package_seeeduino_boards_index.json]
directories:
  data: $PWD/data
  downloads: $PWD/staging
  user: $PWD/user
EOF
bin/arduino-cli --config-file arduino-cli.yaml core update-index
bin/arduino-cli --config-file arduino-cli.yaml core install Seeeduino:nrf52      # Seeed nRF52
        Boards 1.1.x
bin/arduino-cli --config-file arduino-cli.yaml lib install ArduinoJson           # 7.x
cd ..
firmware/build.sh
```

Output	Use
`firmware/xiao_test_tool/build/xiao_test_tool_full.uf2`	**S140 SoftDevice + app — use this one**
`firmware/xiao_test_tool/build/xiao_test_tool.uf2`	app only (only if the board already runs xiao-test-tool)

`build.sh` also regenerates `usage_names.h` from `backend/hid_usage.py`, the single source of key names. In the Arduino IDE the board is *Seeed nRF52 Boards* ▸ *Seeed XIAO nRF52840* (not the “mbed-enabled” core).

3.4 Wireshark (only for passive capture)

See Passive capture.

3.5 Phone access

- Same Wi-Fi as the PC, open `http://<PC-IP>:8000` — works in any phone browser.
- To **install it as an app** (full screen, offline shell) phones need HTTPS: start the bridge with `--https` and install the local CA once — see Web app ▸ Phone.
- Allow TCP 8000 (and 8443 for HTTPS) through the PC’s firewall if the phone can’t connect.

4 · Firmware and flashing

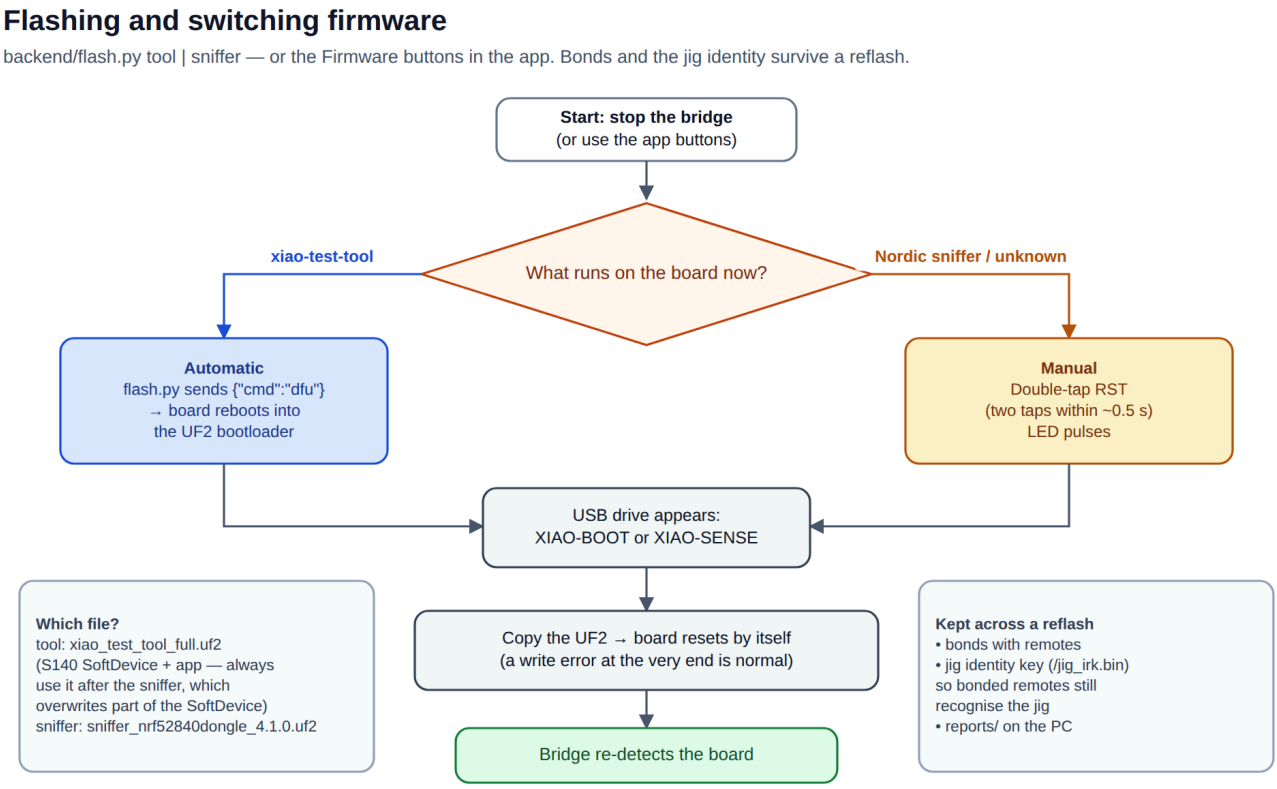
Which firmware?

	xiao-test-tool	Nordic nRF Sniffer 4.1
File	`firmware/xiao_test_tool/build/xiao_test_tool_full.uf2`	`sniffer_nrf52840dongle_4.1.0.uf2`
Role	BLE **central**: pairs with the remote	passive **listener**
Sees key presses	yes, in clear text	no (link is encrypted once bonded)
Voice, battery, GATT, IR	yes	no
Front end	this web app	Wireshark / `ble_probe.py`
Banner in the app	**jig online**	**sniffer image** + flash button
Changes the remote's pairing	yes — bonds it to the jig	no

Full decision chart: Overview ▸ Decision chart.

How flashing works

The XIAO has a **UF2 bootloader**. In bootloader mode it appears as a USB drive named `XIAO-BOOT` (or `XIAO-SENSE` on boards with the Sense bootloader); copying a `.uf2` file onto it flashes it.



Option A — from the web app

Connect ▸ Log ▸ Firmware (bottom of the Connect tab):

- **Flash Nordic sniffer (for Wireshark)** — xiao-test-tool → sniffer. Fully automatic.
- **Re-flash xiao-test-tool** — e.g. after a firmware update.

When the sniffer image is running, the banner shows **Flash xiao-test-tool**; press it and **double-tap RST** on the XIAO when asked. Progress appears as messages; the bridge re-detects the board.

Option B — with the helper

Stop the bridge first (it holds the serial port), then:

```
.venv/bin/python backend/flash.py tool       # install xiao-test-tool
.venv/bin/python backend/flash.py sniffer    # install the Nordic sniffer
```

Starting from	What happens
xiao-test-tool	no button needed: the script sends `{"cmd":"dfu"}`, the board enters the bootloader, the copy runs
Nordic sniffer (or unknown)	the script says *"Double-tap the RST button"* — tap RST twice within ~0.5 s; it waits up to 3 min (`--wait 600` for longer)

Option C — by hand

1. Double-tap **RST** → the `XIAO-BOOT` / `XIAO-SENSE` drive appears.
2. Copy the `.uf2` onto it.
3. The board reboots by itself. A *write error* at the very end is normal — the board resets as the last block lands.

Rules

- **Coming from the sniffer, always use `xiao_test_tool_full.uf2`.** The sniffer overwrites part of the S140 SoftDevice; the full file restores it. The app-only file would leave a board that doesn't start.
- **Kept across a reflash:** bonds with remotes, the jig's identity key (`/jig_irk.bin`, so bonded remotes still recognise the jig), and everything in `reports/` on the PC.
- **One image at a time.** To sniff and test simultaneously, use a second XIAO for the sniffer.
- The bridge can stay running while you flash: it notices the board leaving and coming back.

Checking what runs

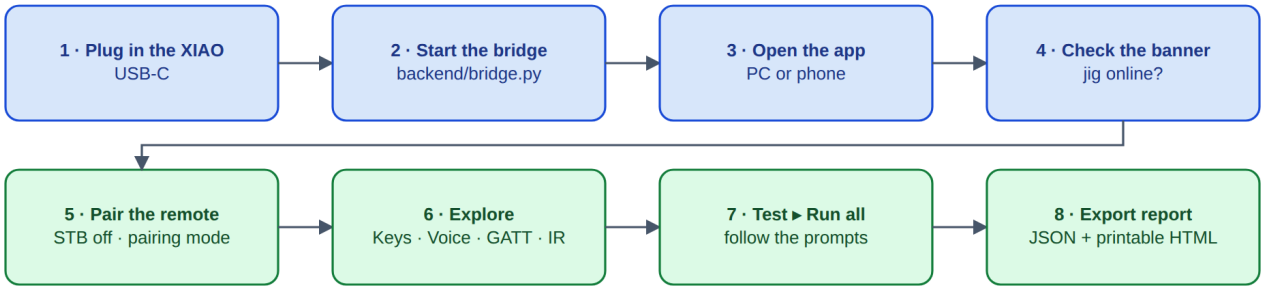
- **Web app banner:** *Board: jig online* + *Firmware: xiao-test-tool 0.2*, or *sniffer image*.
- **Bridge log:** `detected tool firmware on /dev/ttyACM0`, or a warning that the port runs the Nordic image.
- **Serial terminal** (bridge stopped): `.venv/bin/pyserial-miniterm /dev/ttyACM0 1000000`, type `{"cmd":"ping"}` → `{"ev":"hello","fw":"xiao-test-tool","ver":"0.2",…}`.

5 · Quick start

From a fresh board to decoded key presses. Assumes Installation is done.

A test session at a glance

Typical bench routine with the xiao-test-tool firmware (Ch. 5 Quick start).



1 · Plug in the XIAO

USB-C data cable to the PC. If it still runs the Nordic sniffer, that's fine — step 4 fixes it.

2 · Start the bridge

```
cd BT-Sniffer
.venv/bin/python backend/bridge.py
```

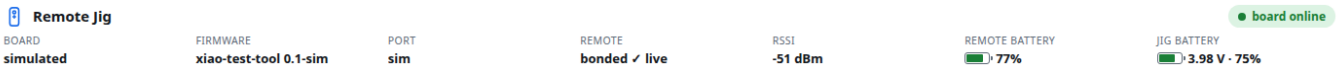
```
bridge: serving web app: http://localhost:8000  http://192.168.1.26:8000
bridge: detected tool firmware on /dev/ttyACM0
```

Leave it running for the whole session; replugging and reflashing are handled automatically.

3 · Open the app

PC: **http://localhost:8000** · phone on the same Wi-Fi: **http://<PC-IP>:8000**.

4 · Check the banner

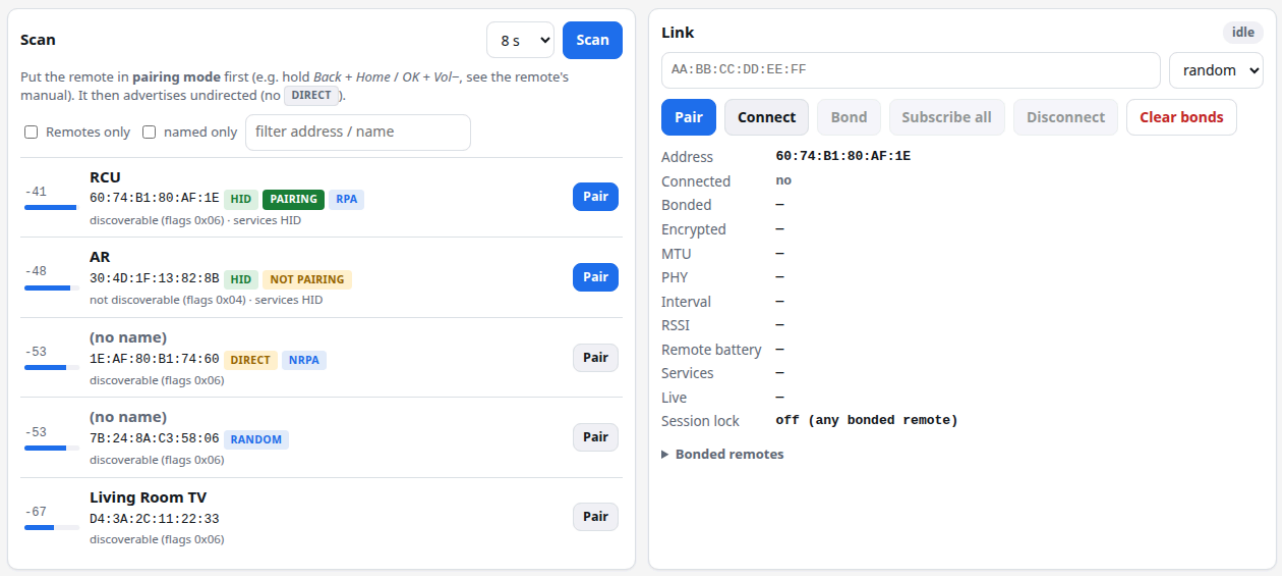


Banner	Meaning	Do
board online · Board **jig online**	xiao-test-tool is running	continue
sniffer image	the Nordic sniffer is flashed	press **Flash xiao-test-tool**, double-tap RST
no board	nothing detected	check the cable; `ls /dev/ttyACM*`; Troubleshooting

Banner	Meaning	Do
offline	the page can't reach the bridge	is `bridge.py` running? same host/port?

5 · Pair the remote

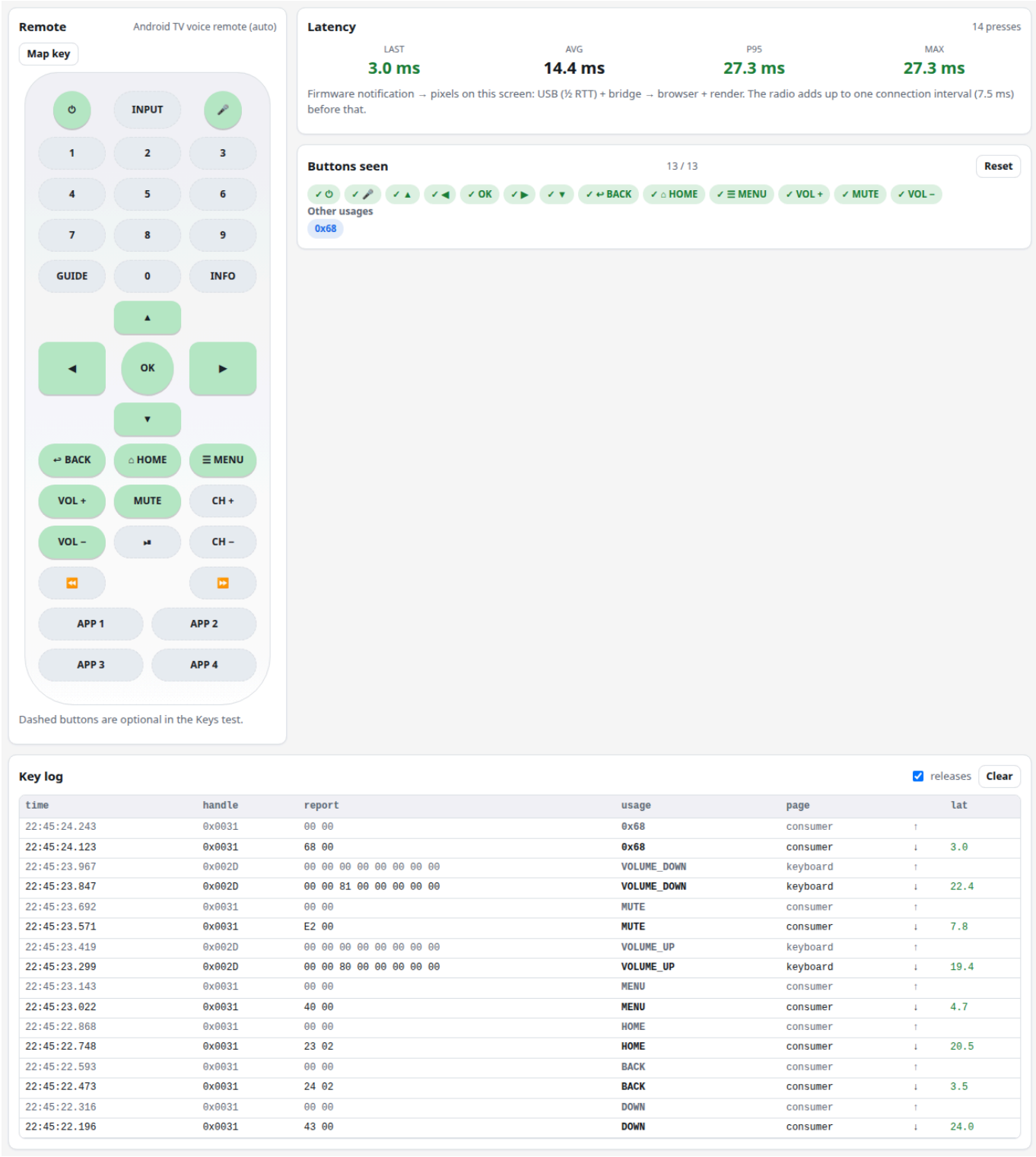
1. **Switch the set-top box off** (unplug it) so the remote can't reconnect to it.
2. **Connect ▸ Scan**.
3. Put the remote into **pairing mode**:
   - Android TV voice remotes: hold **Back + Home** (or **OK + Vol−**) ~5 s until the LED blinks;
   - Fire TV remotes: hold **Home** ~10 s (factory reset first if needed — Ch. 7).
4. The remote appears with **HID** and green **PAIRING** badges. Press its **Connect**.



5. When the stage shows *discovered*, press **Bond**. The stage moves *pairing* → *reading* → ***ready***; the XIAO's blue LED is on while connected.

6 · Press keys

Open **Keys** and press buttons on the remote. Each lights up on the drawing and is listed with its decoded name and latency.



7 · Run the test and export

Test ▸ Run all, follow the prompts, then **Export report** — see Testing and reports.

Next

- Everything on each screen: Web app tour
- Why some remotes need pairing mode every session: Pairing and remotes

No hardware? `.venv/bin/python backend/bridge.py --simulate` runs a simulated jig and remote, including keys, voice, IR and the test runner — useful for training. All screenshots in this manual were taken that way.

6 · Web app tour

The app has a **banner** that is always visible and five **tabs**. All tabs work with the xiao-test-tool firmware. Screenshots were taken with `bridge.py --simulate` (values marked “Simulated” / “SIM-…” are from the simulator).

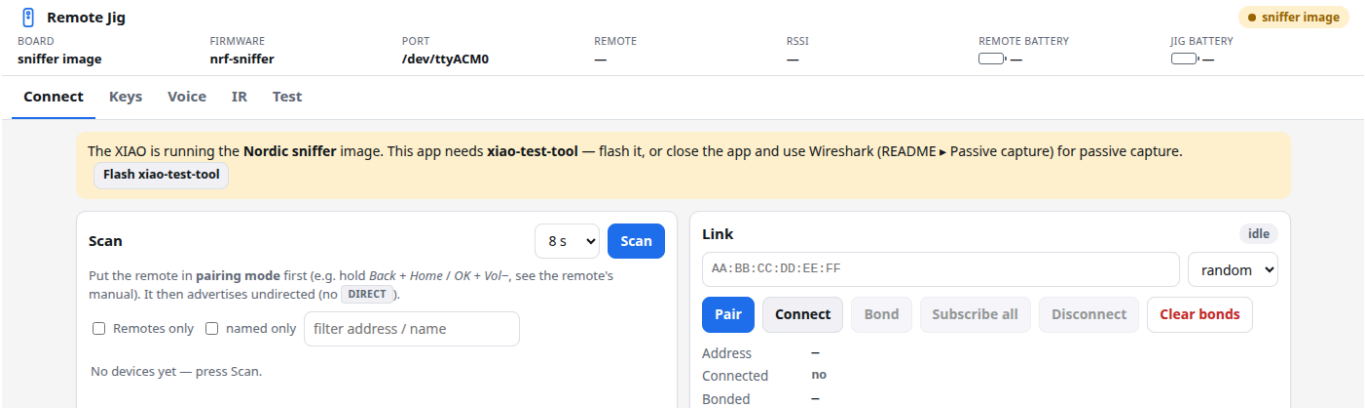
- Banner · Connect · Keys · Voice · IR · Test · On a phone

Banner

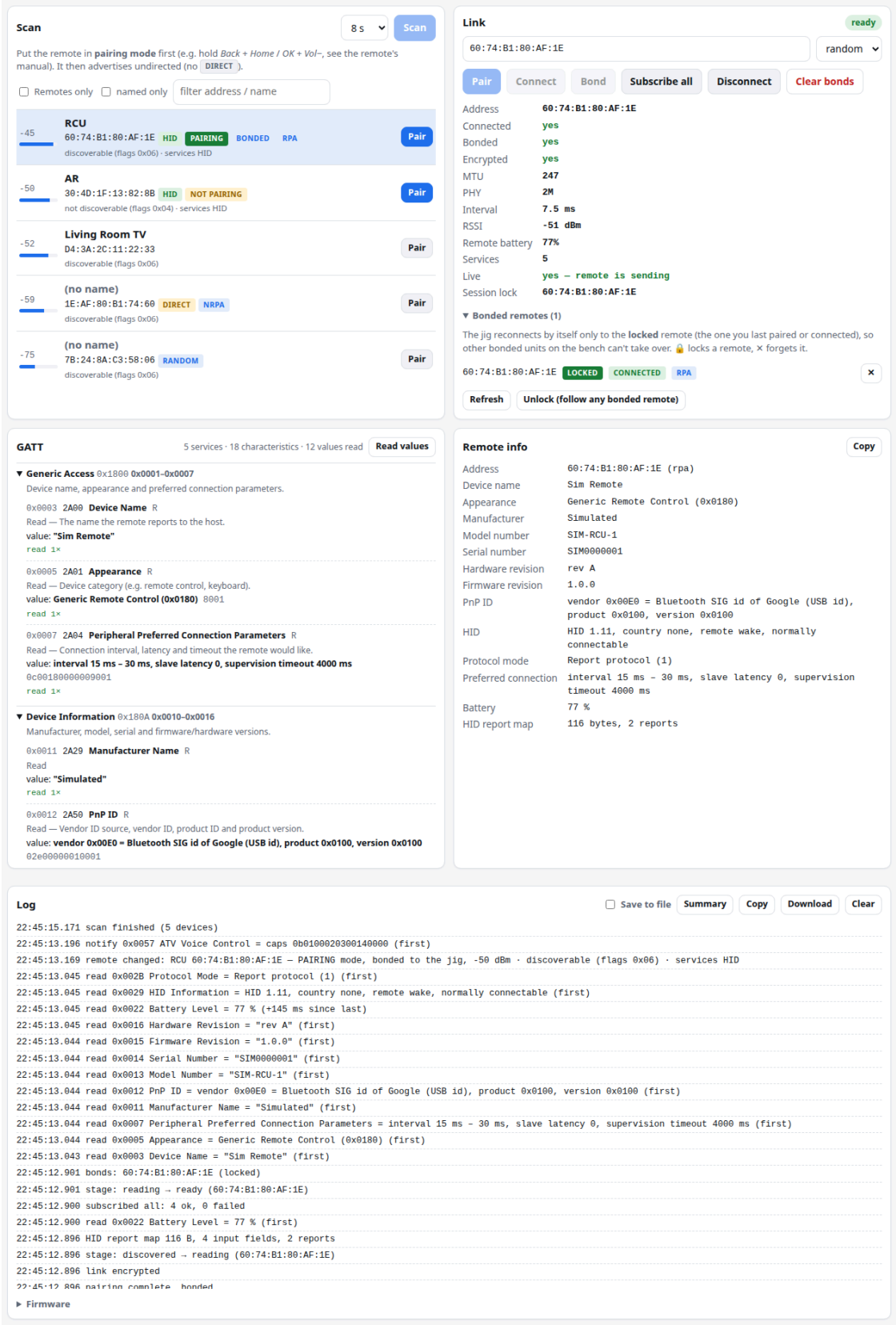


Field	Shows
status pill	**board online** (green), **no board**, **sniffer image**, or **offline** (page can’t reach the bridge)
Board	jig online · simulated · sniffer image · none
Firmware	name and version reported by the board
Port	serial port, e.g. `/dev/ttyACM0`
Remote	link state: stage, **bonded** ✓, or **silent** ⚠ (connected but the remote sends nothing — Ch. 7)
RSSI	signal strength of the remote
Remote battery	the remote’s own Battery Service level. Hover: *read* or *notified*, and when. Dimmed “(last)” while disconnected
Jig battery	jig LiPo voltage and %. **LOW** below 3.4 V

When the board runs the Nordic sniffer image, a hint with a **Flash xiao-test-tool** button appears:



Connect



Scan

Lists nearby BLE devices, strongest first. Badges:

Badge	Meaning
HID	advertises the HID service — a keyboard/remote
BONDED	the jig holds a bond with this remote
PAIRING	its advert flags say *discoverable*: it is in pairing mode and will accept the jig
NOT PAIRING	not discoverable: it is only reconnecting to its own host and will refuse the jig (connect fails with 0x3E)
DIRECT	a directed advert aimed at one specific host
RPA / RANDOM / NRPA	address type (resolvable private, random static, non-resolvable)

Under each name, a decoded line describes the advert: discoverable or not (flags), appearance (e.g. Generic Remote Control), maker (Bluetooth company ID, e.g. Amazon.com Services), advertised services and TX power. Bonded remotes sort first, then HID devices.

Filters: *Remotes only* (HID, remote/keyboard appearance, or bonded to the jig — some remotes advertise neither HID nor an appearance), *named only*, text search. Click a row to fill the address; **Pair** on a row pairs it in one step (below). Pressing Pair on a **NOT PAIRING** remote asks for confirmation first.

Link

Buttons: - **Pair** — the normal way: connect → pair → subscribe in one step. It pairs **fresh** (any old bond with that remote is deleted first) and retries a connection the remote misses (0x3E / timeout) twice. - **Connect** and **Bond** — the same two steps separately, for diagnosis. - **Subscribe all** (re-enable notifications) · **Disconnect** · **Clear bonds** (forget all remotes).

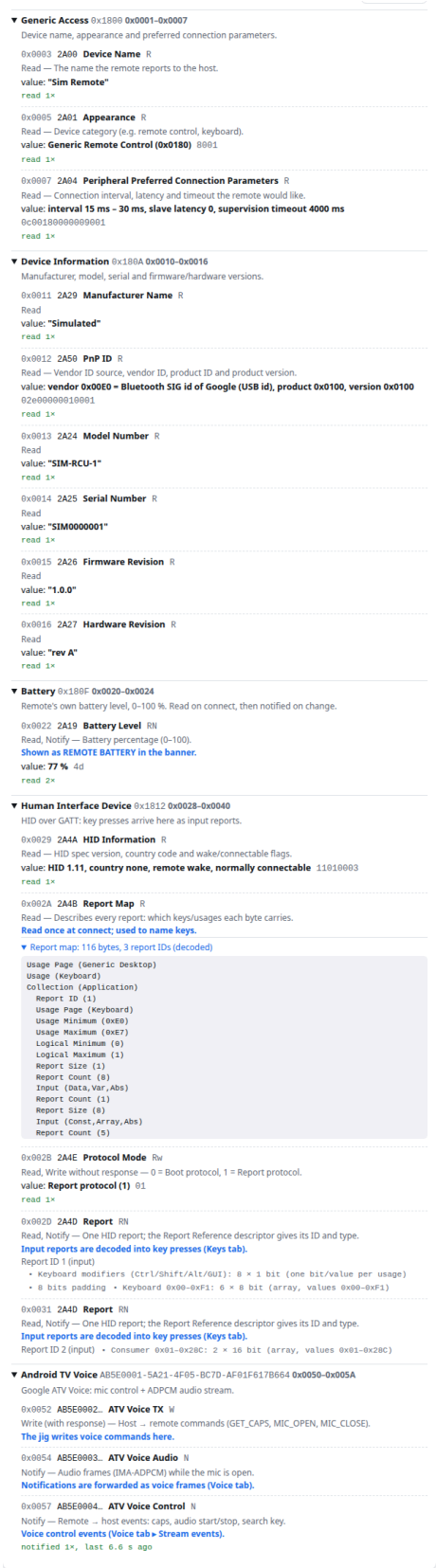
Details: address, connected, bonded, encrypted, MTU, PHY, connection interval, RSSI, remote battery, services, **Live** (yes = the remote is sending; *unknown — press a key* for remotes the jig can't check by itself; **SILENT**) and **Session lock**. The stage label runs *idle → connecting → connected → discovering → discovered → pairing → reading →* ***ready***.

Passkey pairing. A remote that refuses Just Works pairing (code 0x83) gets a second attempt with a passkey: a box shows a 6-digit code — type it on the remote and press OK / Enter.

Bonded remotes (expand under the details) lists every remote bonded to the jig, with **LOCKED**, **CONNECTED** and **RPA** badges. The jig reconnects by itself **only to the locked remote** — the one you last paired or connected — so other bonded units on the bench can't take over a session. 🔒 locks a remote, ✕ forgets one, **Unlock** lets the jig follow any bonded remote again. The lock survives reboots.

GATT

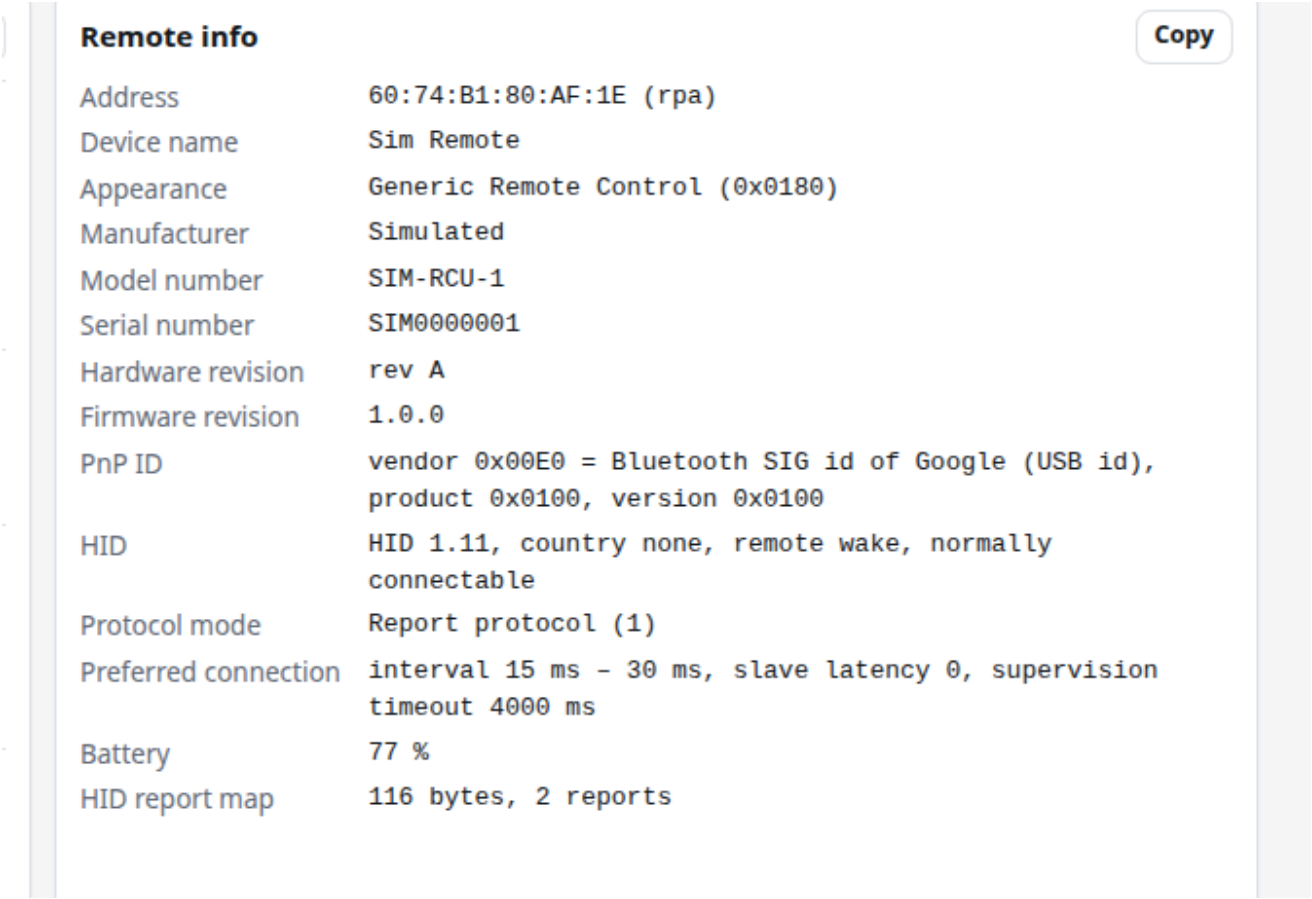
Everything the remote exposes, explained:



- every **service** and **characteristic** with its name and purpose;
- **properties** spelled out — R Read, W Write, w Write without response, N Notify, I Indicate;
- each readable **value**, read once per connection and **decoded** (device name, appearance, preferred connection parameters, Device Information strings, PnP ID, HID information, protocol mode, battery); raw hex beside it;
- the **HID report map** decoded — under each Report: its ID and which keys it carries;
- a **tally** per characteristic: read N×, notified N×, last seen, average interval;
- **Read values** reads everything again.

Vendor services (e.g. Amazon's) are labelled *vendor-specific*; their notifications are forwarded raw.

Remote info



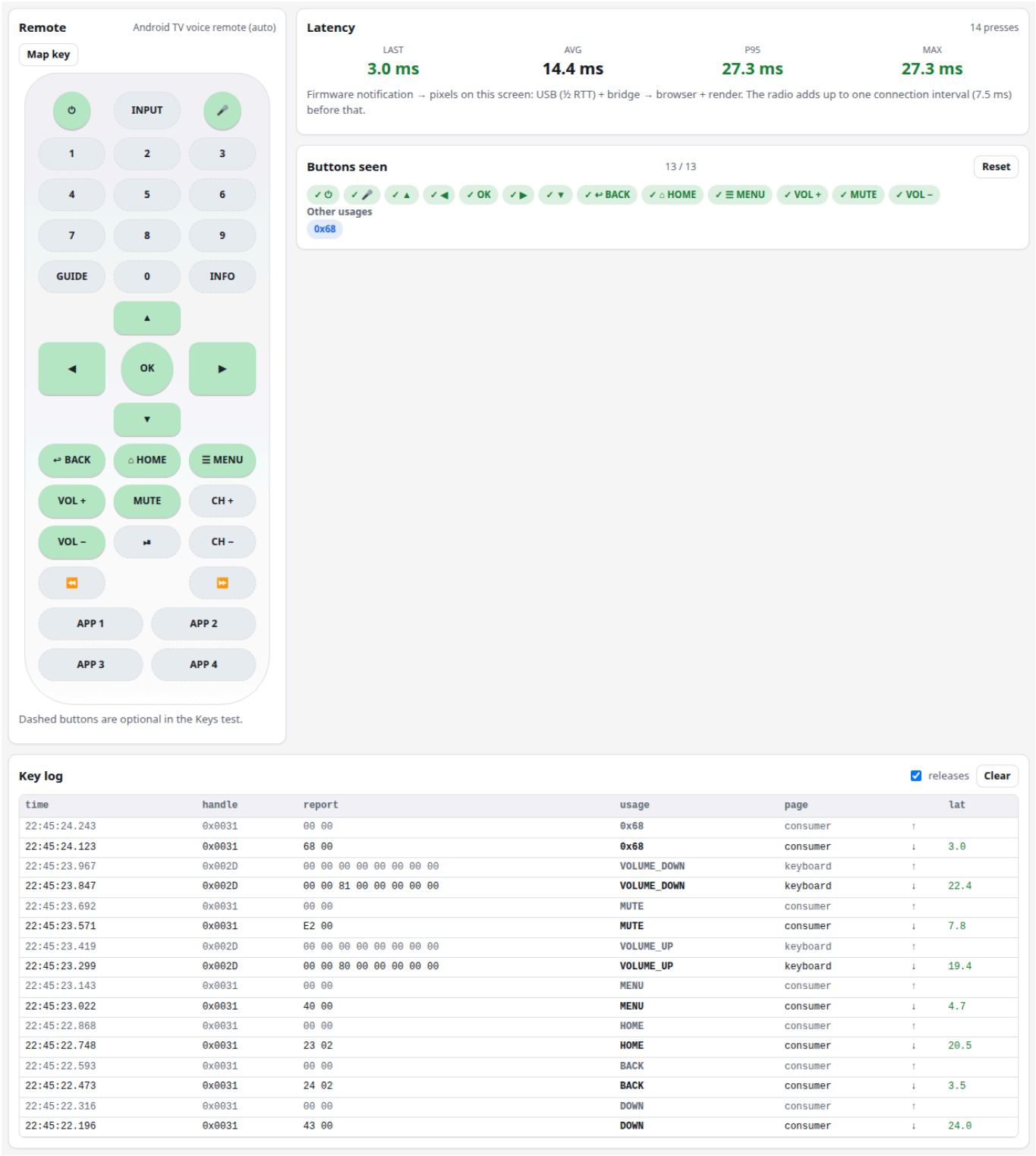
The identity of the connected remote in one list: address, name, appearance, manufacturer, model, serial, hardware/firmware/software revision, PnP ID, System ID, regulatory data, HID version, protocol mode, preferred connection, battery, report-map size. **Copy** puts it on the clipboard. After a disconnect it keeps the last connection's values.

Log



- firmware and bridge messages (connect, pairing, errors), every **stage change** (`stage: discovered → pairing`), **link changes** (connection interval / slave latency / supervision timeout, PHY, data length, MTU), **remote sightings** (first time a remote is seen and whenever it changes: pairing mode on/off, bonded), bond-list changes and passkey prompts;
- one line per **GATT exchange**: `read` (the jig asked) or `notify` (the remote sent it by itself), with the decoded value and the **time since the previous one** on that characteristic — that's how you see how often a remote sends something (e.g. battery updates);
- key presses and voice frames are counted in the GATT tally instead (too frequent to list);
- **Summary** — opens a live table above the log (per characteristic: reads, notifications, last seen, average interval) with its own **Copy**. Close it with **✕ Close**, **Hide summary** or Esc; the log itself is not changed;
- **Save to file** — persistent logging on the PC. Tick it and the **bridge** writes `reports/logs/session_<date-time>.log` in the background (a new file each time you tick it). It keeps going with the browser closed or the phone asleep, and every open page shows the same tick and file name (click the name to open the file). Each line has a timestamp: commands sent, state changes, decoded keys, GATT reads and notifications, battery, voice control, log messages. Voice audio is summarised as one line per stream; keep-alive pings and RSSI-only updates are left out. To start it with the bridge, use `bridge.py --session-log`.
- **Copy** / **Download** — the whole session (up to 20 000 lines, oldest first) with a header (date, jig firmware, port, remote address/name/model/firmware), saved as `remote-log_<address>_<date-time>.txt`;
- **Firmware** — switch images (Ch. 4).

Keys



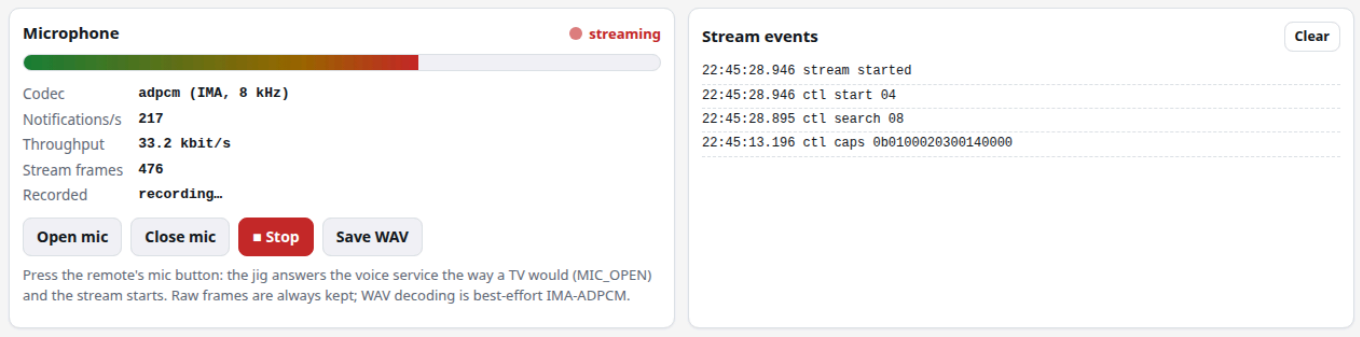
Area	What it does
Remote drawing	the pressed button flashes; buttons already seen turn green. Dashed = optional in the test
Latency	last / avg / p95 / max from the firmware receiving the notification to the row on screen (USB + bridge + network + render). The radio adds up to one connection interval, shown below. < 50 ms is green

Area	What it does
Buttons seen	checklist of expected buttons; **Other usages** collects codes not on the drawing
Key log	time, handle, raw report, decoded usage, HID page, ↓ press / ↑ release, latency. Untick *releases* to hide ↑ rows
Map key	bind a vendor code to a drawn button: press **Map key**, tap the button on the drawing, press the physical key. Saved in this browser

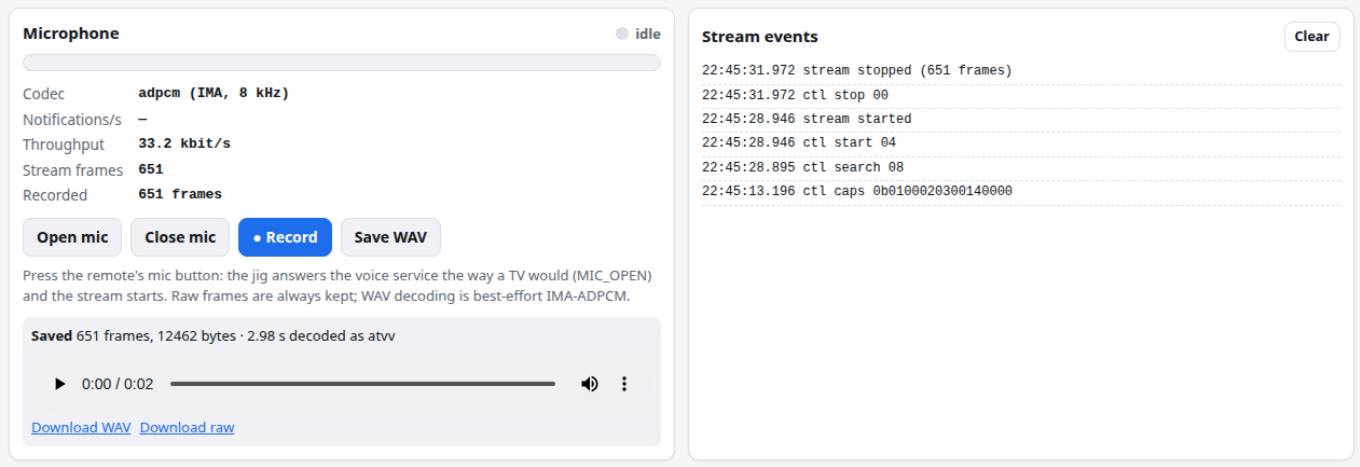
Other HID reports (appears below the key log when needed) lists HID reports whose report map declares **no key usages**: vendor data such as motion/pointer streams (e.g. the LG Magic Remote's 100/s motion reports). They are never shown as keys. The panel gives each stream's count, rate and latest bytes, highlighting bytes that changed — that is how a remote's hidden key codes are found. (LG keys are decoded from this stream by the bridge and appear as normal keys.)

Vendor codes seen so far: Android TV voice remote app keys `0x68`, `0x72` (consumer page); Fire TV remote `0xF1` (keyboard page, probably Back) and `0xA1` – `0xA4` (vendor report 239, probably the app buttons) — confirm on your unit and name them with **Map key**.

Voice



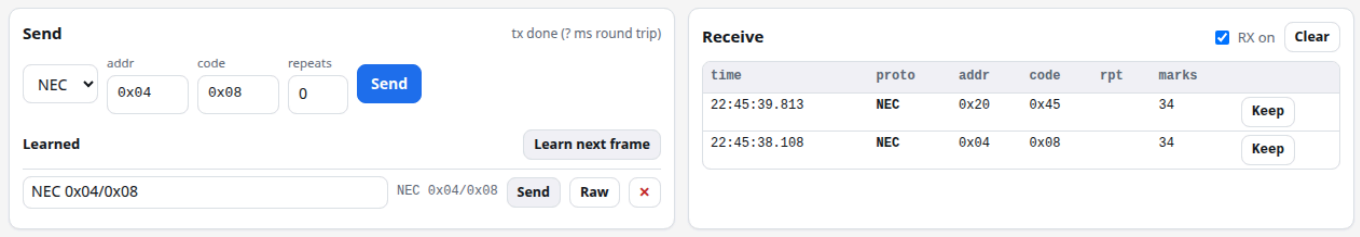
- **Hold the remote's mic button**: the jig answers the Android TV Voice service like a TV, the indicator turns red and the level meter, notifications/s and throughput update live.
- **Open mic** / **Close mic** send MIC_OPEN / MIC_CLOSE (hold-to-talk remotes answer MIC_OPEN with an error — use the button).
- ● **Record** collects frames until ■ **Stop**. **Save WAV** decodes IMA-ADPCM (8 or 16 kHz, detected from the byte rate) and offers a player, **Download WAV** and **Download raw**. The raw file is always kept even if decoding fails. Without a recording, Save uses the last stream.
- Files go to `reports/voice/`.



Fire TV remotes: not supported yet — their mic arrives as a `SEARCH` key and the audio uses Amazon's own protocol (Ch. 7).

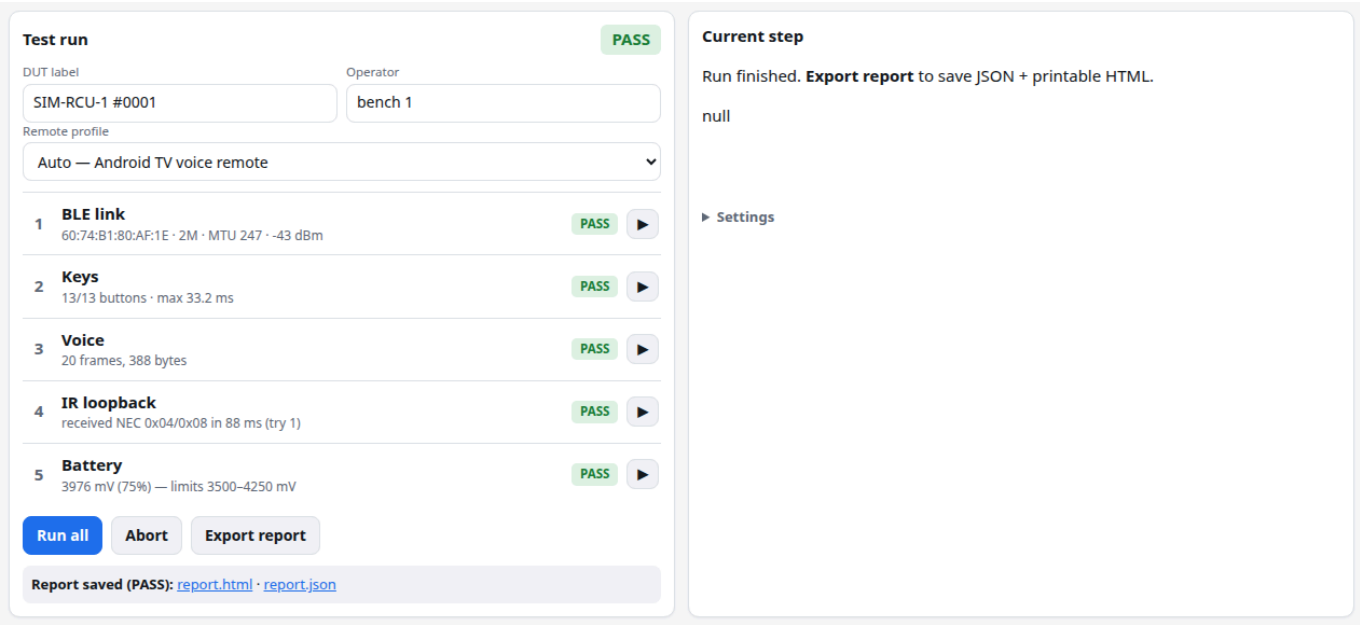
IR

Needs the Unit IR (Hardware).



- **Receive**: every frame — protocol (NEC, SIRC, RC5, UNKNOWN), address, code, repeat flag, mark count. **RX on** toggles the receiver.
- **Learn next frame**, then press a key on any IR remote aimed at the unit: the frame is stored under **Learned** (rename, **Send**, **Raw** replays the exact timings, ✕ deletes). **Keep** on a received row does the same.
- **Send**: NEC or SIRC by address/code with optional repeats; the status shows when it finished. Refused while the jig battery is below 3.3 V.

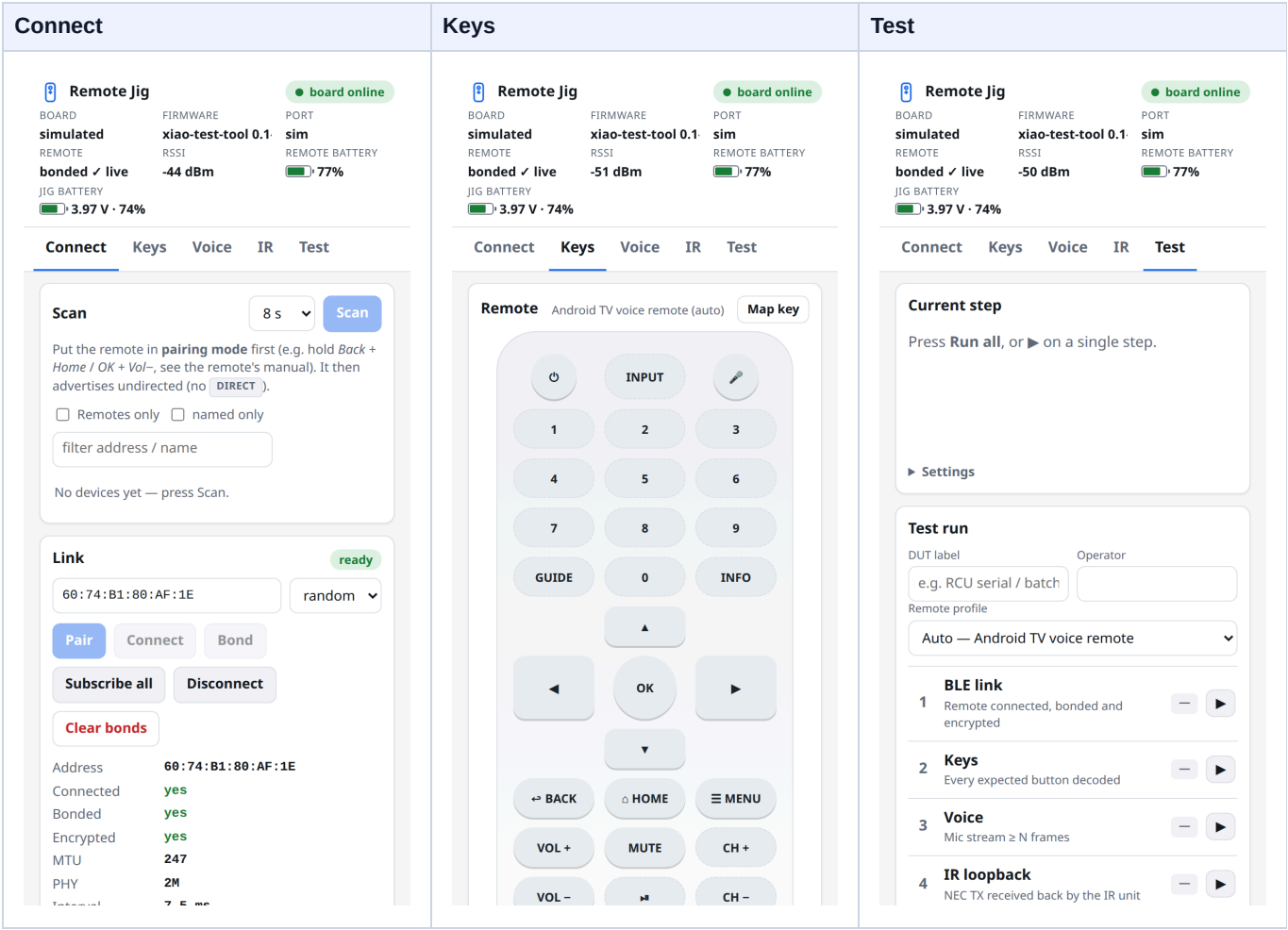
Test



Scripted pass/fail run with report export, with a **Remote profile** per remote model — see Testing and reports.

On a phone

The layout stacks into one column; on the Test tab the *Current step* panel comes first.



Install as an app (full screen, offline shell) — phones require HTTPS:

1. Start the bridge with `--https` (creates a local CA in `.certs/`, serves `https://<PC-IP>:8443`).
2. On the phone open `http://<PC-IP>:8000/ca.crt` and install it:
   - Android: Settings ▸ Security ▸ Encryption & credentials ▸ Install a certificate ▸ CA certificate;
   - iOS: install the profile (Settings ▸ General ▸ VPN & Device Management), then enable it under Settings ▸ General ▸ About ▸ Certificate Trust Settings.
3. Open `https://<PC-IP>:8443` → *Install app* (Chrome) or *Share ▸ Add to Home Screen* (Safari).

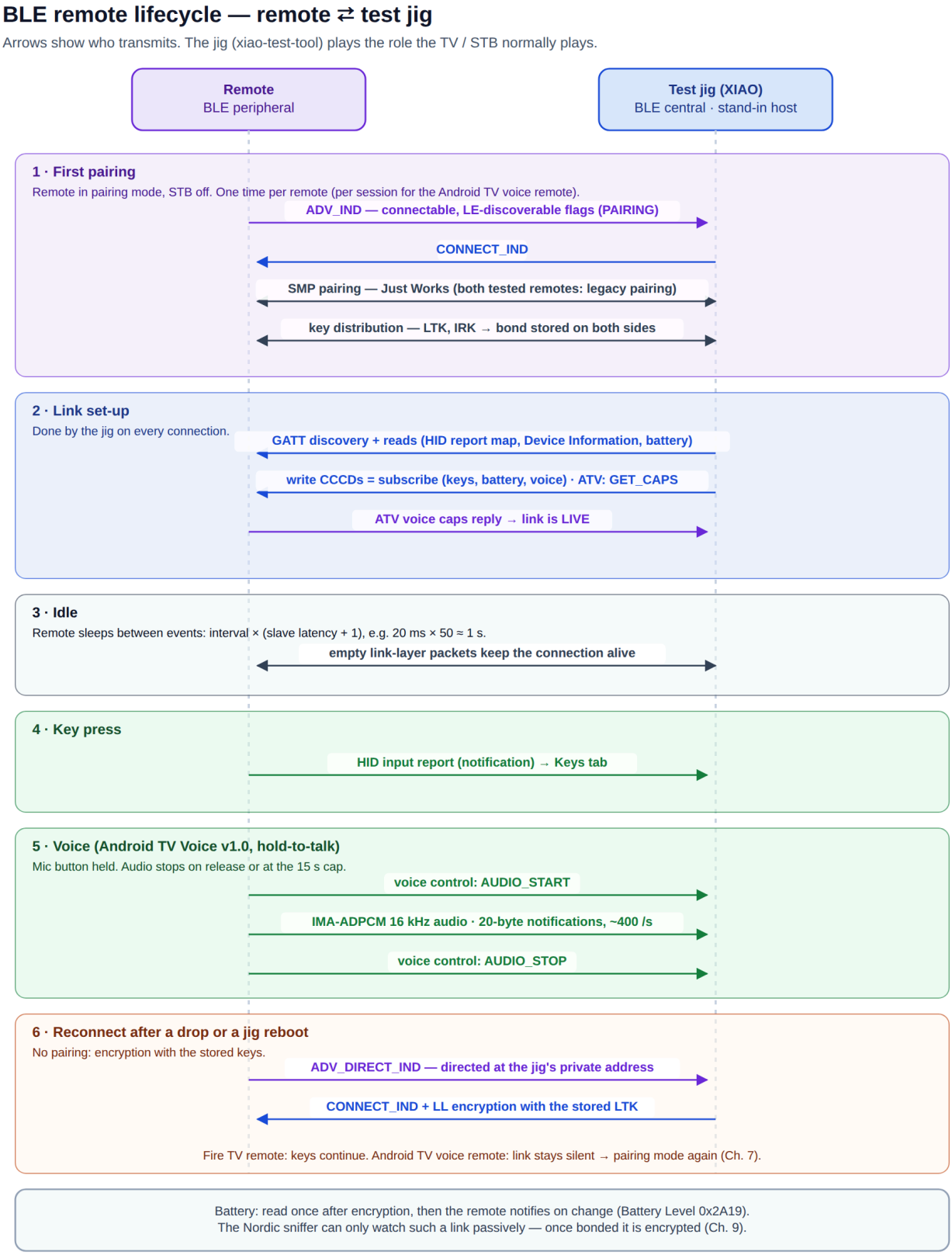
Only install this CA on devices you control; delete `.certs/` to make a new one. The app always loads the latest version from the bridge and falls back to its cached copy only when the bridge is offline.

7 · Pairing and remotes

The jig only sees key presses in clear text because it **pairs with the remote itself**, taking the TV's place. This chapter explains pairing, what happens afterwards, and how different remotes behave.

> **Pairing bonds the remote to the jig.** Many remotes remember only one host; pair the remote with its set-top box again after testing if needed.

What happens over the air



1. **First pairing** — remote in pairing mode advertises (connectable, *discoverable*); the jig connects and pairs (Just Works — no PIN). Both sides store the keys (the *bond*). Both tested remotes used legacy pairing.
2. **Link set-up** (every connection) — the jig reads the GATT table, HID report map, Device Information and battery, subscribes to notifications, and, if the remote has the Android TV Voice service, sends `GET_CAPS`. A working link answers at once.

3. **Idle** — the link stays up; the remote sleeps between connection events.
4. **Key press** — a HID input report notification → Keys tab.
5. **Voice** — mic held: AUDIO_START, ADPCM audio, AUDIO_STOP.
6. **Reconnect** — after a drop or a jig reboot, the remote calls the jig back (*directed* advert to the jig's private address) and the link is encrypted with the stored keys — no pairing.

Pairing a remote

1. **Switch the set-top box off.** Otherwise the remote may reconnect to it instead.
2. **Connect ▸ Scan.**
3. Put the remote into **pairing mode** and keep it within ~1 m:

Remote	Pairing mode
Android TV voice remotes	hold **Back + Home** (or **OK + Vol−**) ~5 s until the LED blinks
Fire TV remotes	hold **Home** ~10 s. Still **NOT PAIRING**? Factory-reset: hold **Left + Menu + Back** 12 s, release, wait 5 s, then hold **Home**
others	see the remote's manual; usually a two-key combination held 3–10 s

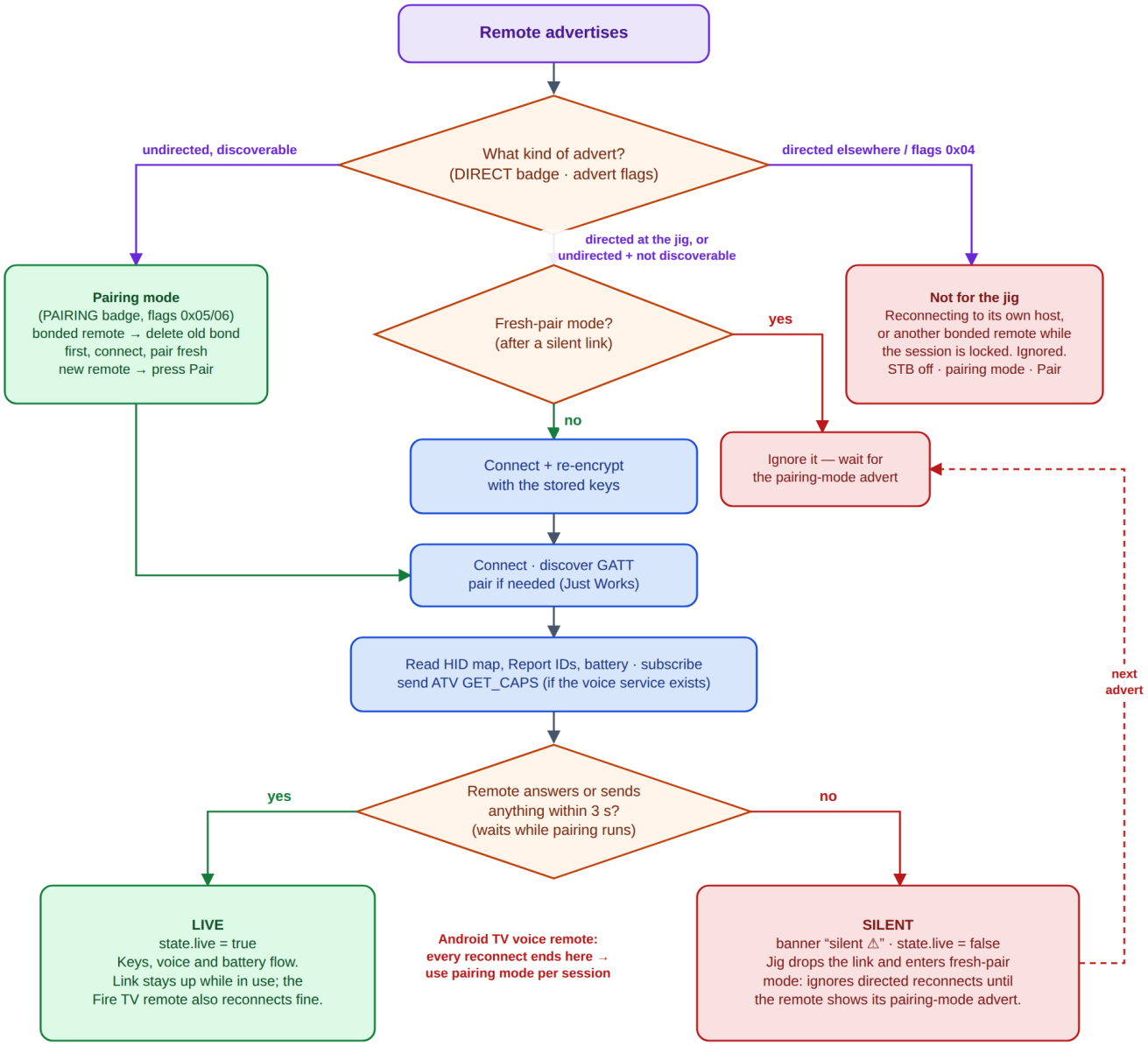
4. Wait for the green **PAIRING** badge — **NOT PAIRING** means it is still trying to reach its own host and the jig's connection will fail (0x3E). Press the row's **Pair**.
5. The jig connects (retrying twice if the remote misses the connection — common), pairs (Just Works; a passkey fallback if the remote insists), reads the remote and subscribes. The log shows each stage.
6. **ready** = bonded ✓, encrypted ✓, report map read, notifications on. Blue LED on. The banner shows **bonded ✓ live** once the remote has sent something (or *press a key* for remotes the jig can't check).
7. The remote is now the session's **locked** remote: the jig follows only it.

Connect + **Bond** still exist for diagnosis; Pair is the same with retries and always pairs fresh.

How the firmware handles links

How the jig connects, pairs and checks a link

Firmware logic in ble_central.cpp (v0.3). Only the session's locked remote is followed; Pair retries missed connections.



The remote advertises…	The jig…
undirected + discoverable (flags 0x05/0x06), **unknown** remote	nothing automatic — press **Pair**
undirected + discoverable, **bonded** remote (pairing mode)	deletes the old bond **before** connecting, then connects and pairs fresh
directed **at the jig**, or undirected + connectable but **not** discoverable (reconnect)	connects and re-encrypts with the stored keys — unless in *fresh-pair mode* (below)
directed at **another host**, or not connectable	ignores it; logs that the remote is reconnecting to its own host
anything from a bonded remote that is **not the locked one**	ignores it; logs “session is locked to another remote”

Rules that keep bonds safe: - Only the **advert** decides (not the scan response), and only its **Flags** say “pairing mode”. A remote waking up with an ordinary undirected advert is reconnected with its keys, not re-paired. - The old bond is deleted **before** a fresh pairing connects: Bluefruit re-encrypts with any stored bond the instant a link opens,

which would otherwise win over the new pairing. - Bonds are deleted by the remote's **identity** address, so remotes using private (RPA) addresses are handled too. - A bond is dropped after a failed re-encryption only for "the remote has no keys for us" reasons (0x13, 0x05, 0x06, 0x0E, 0x3D) — never for a lost radio link (0x08).

Remotes differ on reconnect

Remote	After a drop / jig reboot	What you do
Fire TV remote (30:4D:1F:13:82:8B)	reconnects with the stored keys and **keeps sending** keys and battery	press any key to wake it
Android TV voice remote (60:74:B1:80:AF:1E)	reconnects and encrypts, but **sends nothing**	pairing mode at the start of every session

Silent links

On the Android TV voice remote, only a link made by a **fresh pairing** works. A reconnected link looks healthy — encrypted, subscriptions accepted — but carries no keys, no voice and no reply to the voice handshake. Re-pairing over such a link does not help either. A freshly paired link, on the other hand, stays up for as long as you use it (more than 7 minutes idle was observed).

The firmware detects and handles this automatically (only for remotes with the Android TV Voice service — others are marked live on their first notification):

1. **Check** — ~3 s after `GET_CAPS` (longer while pairing is still exchanging keys) with no reply: `state.live = false`, banner **silent** ⚠, a warning; the jig **drops the link**.
2. **Fresh-pair mode** — the jig ignores the remote's directed reconnects (they would only produce another silent link and keep the remote out of real pairing mode).
3. **Pairing mode** — on the remote's undirected advert, the jig deletes the old bond first and pairs fresh (otherwise the Bluetooth stack would answer with the old keys and the link would be silent again).

In practice: STB off, remote in pairing mode, wait for **bonded** ✓ — and when the banner ever shows **silent** ⚠, just put the remote in pairing mode again. No app buttons needed.

After a jig reboot with **no** bond stored (e.g. after **Clear bonds**), the jig does not scan by itself — use **Scan** → **Connect** → **Bond**.

LG Magic Remote

Tested 2026-10-29 (F4:83:2C:6F:C4:98). Pairs and reconnects normally. It needs special handling for its keys:

- Its HID report map declares every report as an **opaque byte array** (Consumer page, values 0–255, no usage list), so a standard decoder finds no key codes in it.
- It streams **~100 reports/s** of motion data (report 0xFD, 19 bytes: six 16-bit gyro/accelerometer values) for the on-screen pointer. These appear under **Keys ▸ Other HID reports**, not as keys.
- Its **keys travel inside that stream**: byte 16 bit 7 = a key is held, byte 17 = the key code. The codes are **LG's TV infrared command codes**. The bridge decodes them (`backend/lg_magic.py`) into ordinary key events.

Confirmed on the remote	Code
OK, Up, Down, Left, Right	0x44, 0x40, 0x41, 0x07, 0x06
Back, Home, Settings	0x28, 0x7C, 0x43
Volume +/−, Channel +/−	0x02, 0x03, 0x00, 0x01

Other codes seen but not yet tied to a button: 0x0C, 0x3E, 0x48, 0x56, 0x5C, 0x81, 0x8B, 0xA1, 0xAE (likely mic, mute, input, number pad and app shortcuts). They show as `LG_0x..`; bind them with **Keys ▸ Map key**, or add them to `LG_KEYS` in `backend/lg_magic.py`. LG voice is not supported.

Fire TV voice

The Fire TV remote's mic button arrives as a `SEARCH` key (consumer usage 0x221). Its audio uses Amazon's own GATT services, whose start command is not public:

- `FE151500-5E8D-11E6-8B77-86F30CA893D3`
- `5DE20000-5E8D-11E6-8B77-86F30CA893D3`
- `CFBFA000-762C-4912-A043-20E3ECDE0A2D`

The jig subscribes to them and forwards any notification as raw voice frames, but the remote waits for a command it doesn't receive. The way forward is a Wireshark capture of a real Fire TV ⇄ remote voice session that includes the pairing (Passive capture).

Bonds and the session lock

Connect ▸ Link ▸ Bonded remotes lists every remote bonded to the jig (up to 16). The jig follows only the **locked** remote — set automatically by Pair / Connect, or with 🔒. With several units on the bench, a stray key press on an earlier unit is ignored (and logged) instead of taking over the jig; if a different remote does connect during a test run, the run and the report flag it.

- ✕ forgets one remote; **Clear bonds** forgets all (and unlocks).
- **Unlock** lets the jig follow any bonded remote again.
- Bonds and the lock survive reboots and reflashing.

Addresses

Shown most-significant byte first (standard Bluetooth order). Old `ble_probe.py` output and `BT-sniffed.txt` printed them byte-reversed (e.g. `1E:AF:80:B1:74:60` for `60:74:B1:80:AF:1E`); fixed.

8 · Testing and reports

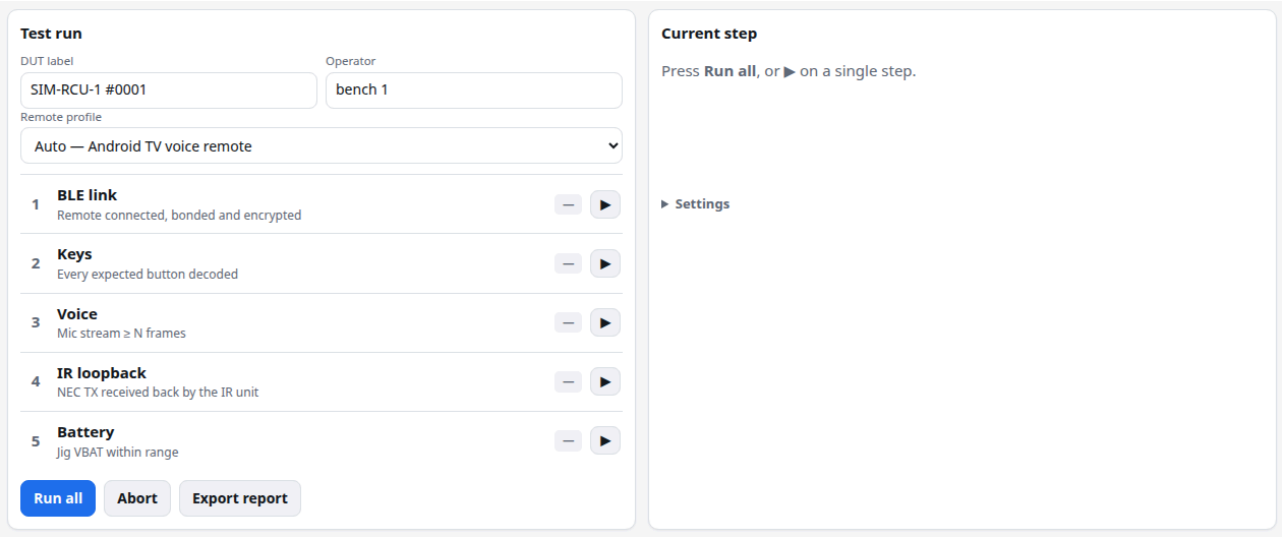
The **Test** tab runs a scripted pass/fail check of one remote (the *DUT*, device under test) and the jig, and exports a report.

Before you start

- Remote paired: banner shows **bonded** ✓ (Pairing).
- For the IR step: Unit IR wired and a white card or other reflective surface ready.

Running

1. **Test** tab → fill **DUT label** (e.g. the remote's serial number — **Connect ▸ Remote info** has it) and **Operator**.



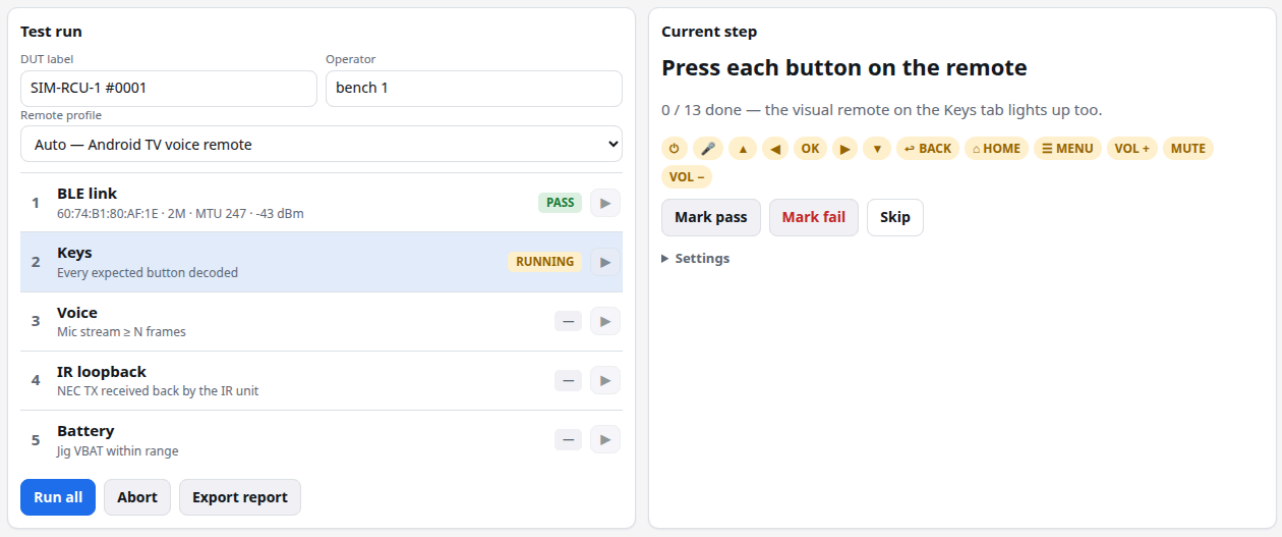
2. Check the **Remote profile** — it decides which buttons are expected and how voice is tested. *Auto* picks it from the connected remote's GATT services:

Profile	Expected buttons	Voice
Android TV voice remote	the drawing's solid buttons (power, mic, D-pad, OK, back, home, menu, volume, mute)	Android TV Voice stream
Fire TV remote	mic, D-pad, OK, back (0xF1), home, menu, ⏪ ⏯ ⏩, volume, mute; app keys 0xA1–0xA4 mapped	*n/a* (Amazon's protocol is not supported yet)
LG Magic Remote	D-pad, OK, back, home, volume, channel (keys decoded from its data stream)	*n/a*
Generic BLE HID remote	D-pad, OK, back, home, volume	*n/a*

   The Keys tab shows the active profile next to the drawing.

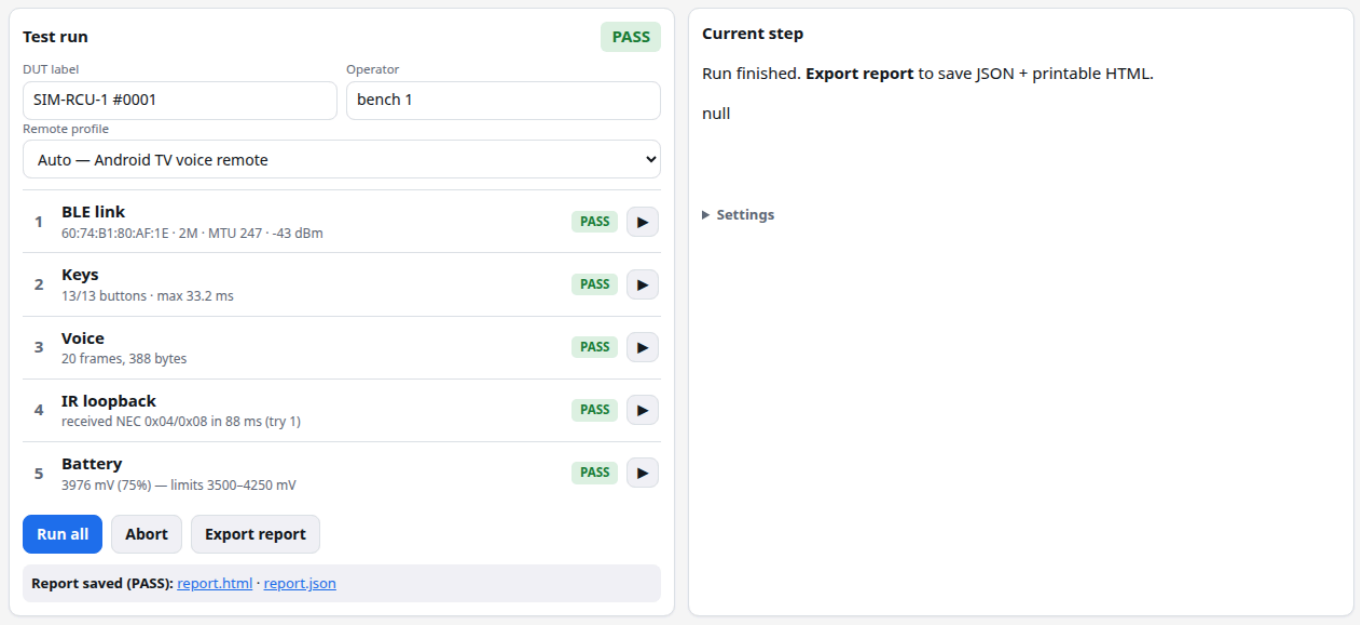
3. Optional: **Settings** (in the *Current step* panel) — limits and timeouts, below.

4. **Run all**. Follow the **Current step** panel (shown first on phones):



#	Step	What you do	Pass when
1	BLE link	nothing	remote connected and encrypted, and not silent
2	Keys	press every button listed (they tick off as they arrive)	every expected button decoded, and max latency ≤ limit
3	Voice	hold the mic button and speak	a voice stream of ≥ N frames arrives (*n/a* — skipped — for profiles without supported voice)
4	IR loopback	hold a white card a few cm in front of the Unit IR (TX and RX share it)	the transmitted NEC frame is received back
5	Battery	nothing	jig battery voltage within limits

- Each step also offers **Mark pass**, **Mark fail** and **Skip** for operator judgement; ▶ reruns one step; **Abort** stops the run.
- If a **different remote** connects during the run, a warning appears and the report records the address change.
- Run start and finish (with the verdict) are written to the **session log** as `note` lines, as is every saved report — so a report and its log always match.
- Verdict: **PASS** when every step that ran passed, **FAIL** if any failed. **Skipped** steps don’t count either way (the report lists them as *skip*); a run where everything was skipped is *incomplete*.



Settings

Setting	Default	Used by
Battery min / max mV	3500 / 4250	Battery
Voice min frames	20	Voice
Voice timeout s	20	Voice
Keys timeout s	180	Keys
Key latency max ms	50	Keys
IR addr / code	0x04 / 0x08	IR loopback
IR timeout ms / tries	1500 / 3	IR loopback

Exporting the report

Export report → the bridge writes `reports/report_<date-time>.json` and `.html`; links appear under the steps. Open the HTML and print it (or *Save as PDF*) for the record. The JSON holds the same data for scripts: `title`, `started` / `finished`, `jig` (bridge and firmware info), `dut` (label, address, link details, battery, **profile**, **identity** — manufacturer, model, serial, hardware / firmware / software revision, PnP ID, System ID, HID info … as in Connect ▸ Remote info — and **address_changes**), `operator`, `notes` (address-change warning, session-log file name), `verdict`, and `steps[]` with `id`, `name`, `status` (pass / fail / skip), `detail`, `metrics` (latencies, frame counts, voltages) and timestamps.

Other files the jig writes:

File	From
`reports/voice/voice_<date-time>.wav` / `.bin`	Voice ▸ Save WAV (decoded / raw)
`remote-log_<address>_<date-time>.txt`	Connect ▸ Log ▸ Download (your browser's download folder)
`reports/logs/session_<date-time>.log`	Connect ▸ Log ▸ **Save to file** (persistent, written by the bridge)

Tips for production use

- Use one browser per bench; the bridge serves several, but all see the same jig.
- **Remote info ▸ Copy** gives model, serial and firmware for your records.
- Put each new DUT in pairing mode with the STB off; the jig pairs it fresh.
- **Clear bonds** between DUTs if you don’t want the jig to keep following earlier remotes.

9 · Passive capture (Nordic sniffer + Wireshark)

Use this to **watch a real remote ⇄ STB link** without pairing the remote to anything. It is an occasional diagnostic and **does not use the web app**.

When to use it — and its limit

Good for: connection set-up, reconnect behaviour, timing and connection parameters, PHY switches, packet sizes, and — if you capture the pairing — the traffic itself with legacy pairing.

Not good for: reading key presses of an already-bonded remote. Once bonded, the link is **encrypted**. With LE Secure Connections that can't be undone; with legacy pairing, Wireshark can decrypt only if the capture contains the pairing. To read keys, use the jig.

Flash the sniffer image

```
.venv/bin/python backend/flash.py sniffer        # automatic from xiao-test-tool (stop the
        bridge first)
```

or **Connect ▸ Log ▸ Firmware ▸ Flash Nordic sniffer (for Wireshark)** in the app, or by hand: double-tap RST → drag `sniffer_nrf52840dongle_4.1.0.uf2` onto `XIAO-BOOT` / `XIAO-SENSE`. See Ch. 4.

If the bridge keeps running, the app shows **sniffer image**; the bridge does not hold the port, so Wireshark can use it.

Install the plugin

Once per PC. Wireshark ▸ Help ▸ About ▸ Folders ▸ *Personal Extcap path* shows the folder.

```
mkdir -p ~/.local/lib/wireshark/extcap
cp -r extcap/nrf_sniffer_ble.py extcap/nrf_sniffer_ble.sh extcap/SnifferAPI
        ~/.local/lib/wireshark/extcap/
chmod +x ~/.local/lib/wireshark/extcap/nrf_sniffer_ble.sh
        ~/.local/lib/wireshark/extcap/nrf_sniffer_ble.py
pip install pyserial      # for the Python that Wireshark runs the plugin with
```

Capture

1. Restart Wireshark → choose the interface **nRF Sniffer for Bluetooth LE**.
2. **View ▸ Interface Toolbars ▸ nRF Sniffer for Bluetooth LE**: pick the remote's address as the device to follow and enable following on all PHYs (the link switches to 2M PHY after connecting).
3. Choose the remote **before** it connects — e.g. select it, then power-cycle the STB or press a key.
4. To capture a pairing (for decryption with legacy pairing): start following, then pair the remote with its STB again.

Useful display filters: `btle.advertising_header.pdu_type == 0x05` (CONNECT_IND), `btle.control_opcode` (link-layer control), `btatt` (GATT), `btsmp` (pairing).

When done, flash xiao-test-tool back (Ch. 4).

Terminal alternative: `ble_probe.py`

```
.venv/bin/python ble_probe.py                                  # scan, choose a device, follow it
.venv/bin/python ble_probe.py --address 60:74:B1:80:AF:1E
.venv/bin/python ble_probe.py --name RCU --raw
```

Lighter than Wireshark but it does not follow PHY and channel-map updates, so it goes quiet after a connection is set up. It decodes with `backend/decode.py` (three bugs of the original are fixed: packet types read from the wrong bytes, byte-reversed addresses, swapped master/slave direction labels).

10 · Troubleshooting

Messages with a code explain it themselves, e.g. `disconnected (reason 0x3E = Connection Failed to Be Established: the remote never answered — …)`. Full tables: Reference ▸ Reason and error codes.

Banner and connection

Symptom	Cause and fix
Banner **offline**	The page can't reach the bridge. Is `backend/bridge.py` running? Is the page from the same host/port (default 8000)? On a phone: same Wi-Fi, PC firewall allows TCP 8000.
no board	Plug in the XIAO with a data cable; check `ls /dev/ttyACM*`. *Permission denied* → Installation 3.2. Just double-tapped RST? The board is in the bootloader — flash it or press RST once.
sniffer image	The Nordic sniffer is flashed. Press **Flash xiao-test-tool** in the banner and double-tap RST when asked (Ch. 4), or use Wireshark (Ch. 9).
Bridge: *address already in use*	Another bridge is still running on port 8000. Stop it (`pgrep -af backend/bridge.py`, then `kill <pid>`), or use `--http-port 8080`.
The page shows an old layout after an update	Reload once. (Versions before this manual cached the UI and needed two reloads.)

Pairing and links

Symptom	Cause and fix
Remote not in the scan list	*Remotes only* ticked and the remote advertises no HID info — untick it. Or: not in pairing mode (pairing mode times out after ~30–60 s — retry), or connected to something else. Switch the STB off.
Remote listed only with **DIRECT**	It is reconnecting to a host it is bonded with. Put it in pairing mode.
NOT PAIRING badge / connects then *disconnected (reason 0x3E)* at once	The remote is only reconnecting to its own host and ignores the jig. Unplug that host and use real pairing mode; a Fire TV remote may need a factory reset first (Ch. 7).
connect timed out	The remote stopped advertising, or the advert was directed elsewhere. Retry in pairing mode.
pairing failed	Put the remote back in pairing mode and press **Pair** (it pairs fresh).
pair: gave up — is the remote in pairing mode?	Pair tried three connections and the remote answered none. Check for the **PAIRING** badge; pairing mode times out after ~30–60 s.
A **passkey** box appears	The remote refused Just Works pairing (0x83). Type the 6 digits on the remote and press OK / Enter.

Symptom	Cause and fix
Log: *ignoring bonded remote …: the session is locked to another remote*	Another bonded unit woke up. Intended — the jig stays with the locked remote. To switch, **Pair** the new one, or 🔒 / **Unlock** in Bonded remotes.
A remote has no **PAIRING** / **NOT PAIRING** badge	It advertises neither HID nor an appearance and isn't bonded yet. Untick *Remotes only* and look for its name or address.
Banner **bonded ✓ · press a key**	The link is up but the jig can't tell by itself if the remote sends (no Android TV voice service). Press a key: it turns **live**.
Banner **silent ⚠** / keys stop	The remote only sends on a freshly paired link (Android TV voice remote). Put it in pairing mode (STB off); the jig re-pairs by itself (Ch. 7).
Fire TV remote: keys stop after a while	It dropped the link to save power. Press any key — it reconnects within a second.
Log: *bonded remote is advertising directed to XX:… which is not the jig*	The remote is trying to reach another host (normally the STB). Switch the STB off; use pairing mode.

Keys, voice, battery

Symptom	Cause and fix
Thousands of "keys" per second (fake key flood)	Fixed in firmware 0.3: reports without key usages (motion streams) are no longer decoded as keys. Re-flash the jig. They now appear under **Keys ▸ Other HID reports**.
Key shows as `LG_0x…`	An LG key code not in the table yet. **Map key** binds it; or add it to `LG_KEYS` in `backend/lg_magic.py`.
Key shows as `0x…`	A vendor-specific code. **Keys ▸ Map key** to bind it, or add it to `backend/hid_usage.py` and run `firmware/build.sh` to update the firmware's names.
No voice	Hold the mic button (don't just tap). **Connect ▸ GATT** must show *Android TV Voice*. Try **Voice ▸ Open mic**. Fire TV remotes: not supported yet (Ch. 7).
WAV only a fraction of a second long	Fixed in the current bridge (late packets after the 15 s cap used to replace the recording). Update and use **Download raw** for old captures.
Remote battery shows "—"	The remote has no Battery Service, or it hasn't been read yet — press **Read values**.
Jig battery ~4.2 V with no cell	Normal: the VBAT pin sees the charger output.
Jig battery **LOW** / IR send refused	The cell is below 3.4 V / 3.3 V. Charge over USB-C (Ch. 2).

IR

Symptom	Cause and fix
Nothing received	Check the wiring (IR_RX → D0, power from 3V3), that **RX on** is ticked, and aim the remote at the unit's receiver.
Loopback test fails	Hold a white card 2–5 cm in front of the unit; avoid direct sunlight; check `ir_tx_done` in the IR tab.

Flashing

Symptom	Cause and fix
Helper waits forever	Double-tap RST faster (two taps within ~0.5 s). The drive may be named `XIAO-SENSE`; the helper handles both.
Board doesn't start after flashing the tool from the sniffer	Use `xiao_test_tool_full.uf2` (with the SoftDevice), not the app-only file.
Write error at the end of a manual copy	Normal: the board resets as soon as the last block lands.

Collecting information for a bug report

1. **Connect ▸ Log ▸ Download** — the session log with remote details.
2. **Connect ▸ Remote info ▸ Copy**.
3. The bridge's console output.
4. If it's about the radio link: a Wireshark capture (Ch. 9).

11 · Reference

Bridge command line

```
.venv/bin/python backend/bridge.py [options]
```

Option	Env var	Default	Purpose
`--port`	`BRIDGE_PORT`	auto-detect	serial port of the XIAO
`--http-port`	`BRIDGE_HTTP_PORT`	8000	web app + WebSocket
`--bind`	`BRIDGE_BIND`	0.0.0.0	listen address (0.0.0.0 = reachable from the LAN)
`--baud`	`BRIDGE_BAUD`	1000000	serial speed
`--https`, `--https-port`	`BRIDGE_HTTPS_PORT`	off, 8443	HTTPS with a local CA (for installing on phones)
`--reports`	`BRIDGE_REPORTS`	`./reports`	where reports and voice captures go
`--simulate`		off	simulated jig + remote, no hardware
`--session-log`	`BRIDGE_SESSION_LOG=1`	off	start the persistent session log at launch (`reports/logs/session_<date-time>.log`)
`--log-level`		INFO	DEBUG shows serial traffic

Other tools:

Command	Purpose
`backend/flash.py tool | sniffer | file.uf2 [--port P] [--wait S]`	flash via the UF2 bootloader (Ch. 4)
`firmware/build.sh`	build the firmware → `firmware/xiao_test_tool/build/*.uf2`
`ble_probe.py [--address A] [--name N] [--raw]`	passive capture in a terminal (sniffer image)
`backend/hid_usage.py --emit-c`	regenerate `usage_names.h` (done by `build.sh`)

HTTP endpoints (bridge)

Path	What
`/`	the web app
`/ws`	WebSocket: events out, `{"cmd":…}` in
`/api/status`	bridge status JSON
`/api/report` (POST)	save a report object → JSON + HTML
`/files/…`	`reports/` (reports, voice WAV/raw, `logs/` session logs)
`/ca.crt`	the local CA certificate (with `--https`)

Serial protocol (xiao-test-tool)

One JSON object per line, UTF-8, `\n`-terminated, 1 000 000 baud. The bridge forwards everything to the app adding `"src":"tool"`; events it re-sends from its cache to a new page carry `"cached":true`. Talk to the firmware directly (bridge stopped): `.venv/bin/pyserial-miniterm /dev/ttyACM0 1000000`.

Commands

Command	Purpose
`{"cmd":"ping"}`	reply `hello`
`{"cmd":"scan","sec":6}` / `{"cmd":"scan_stop"}`	scan (`sec: 0` = until stopped)
`{"cmd":"connect","addr":"…","atype":"public"}`	connect (`public` / `random`)
`{"cmd":"pair","addr":"…","atype":"public"}`	one step: connect → pair fresh → subscribe; retries a missed connection twice; locks the session to this remote
`{"cmd":"bond"}`	pair and bond (Just Works; passkey fallback on 0x83)
`{"cmd":"lock","addr":"…"}` / `{"cmd":"unlock"}`	auto-reconnect only to this remote / to any bonded remote (persisted)
`{"cmd":"bonds"}`	emit the bond list
`{"cmd":"bond_delete","addr":"…"}`	forget one remote
`{"cmd":"disconnect"}`	drop the link
`{"cmd":"subscribe","what":"all"}`	notifications: `hid` / `voice` / `battery` / `all`
`{"cmd":"battery"}`	jig battery + the remote's Battery Level
`{"cmd":"state"}`	emit `state`
`{"cmd":"info"}`	re-send `gatt`, `hid`, `state` for the current link
`{"cmd":"gatt_read","handle":"0x0016"}`	read a characteristic
`{"cmd":"gatt_write","handle":"…","bytes":"0c0001","rsp":true}`	raw write
`{"cmd":"mic","on":true}`	ATV MIC_OPEN / MIC_CLOSE
`{"cmd":"repair"}`	pair again on the live link
`{"cmd":"clear_bonds"}`	forget all remotes
`{"cmd":"ir_rx","on":true}`	IR receiver on/off
`{"cmd":"ir_tx","proto":"NEC","addr":"0x04","code":"0x08","repeats":0}`	send NEC / SIRC
`{"cmd":"ir_tx_raw","carrier":38000,"timings":[9000,4500,…]}`	send raw mark/space µs (≤ 200)
`{"cmd":"mode","value":"both"}`	`ble` / `ir` / `both`
`{"cmd":"dfu"}`	reboot into the UF2 bootloader

Events

Event	Content
`hello`	`fw`, `ver`, `roles`, jig `addr`, `bonds`, `mode`
`scan`	`addr`, `atype`, `rssi`, `name`, `hid`, `appearance`, `bonded`, `directed` (+ `target`, `to_jig`), `connectable`, `adv` (advertising data hex — carries the Flags), `rsp` (scan response hex)
`scan_done`	`found`
`state`	`conn`, `stage`, `addr`, `bonded`, `secured`, `mtu`, `phy`, `rssi`, `interval_ms`, `services`, `ntf`, `live`, `auto_reconnect`, `lock`, `bonds`
`link`	`what`: `conn_params` (`interval_ms`, `latency`, `timeout_ms`) · `phy` (`tx`, `rx`, `status`) · `data_length` (`tx_octets`, `rx_octets`) · `mtu` (`mtu`, `remote_mtu`)
`bonds`	`list[]`: `addr` (identity), `atype`, `rpa`, `locked`, `connected`
`passkey`	`code` — 6 digits to type on the remote (passkey pairing)
`gatt`	services → characteristics: `uuid`, `handle`, `props`, `role`, `report_id`, `report_type`
`hid`	`map` (hex), `map_len`, `fields`, `reports`
`gatt_read`	`handle`, `ok`, `bytes`
`key`	`handle`, `report`, `usage`, `usage_id`, `page`, `rid`, `down`, `t`
`battery`	jig: `mv`, `pct`, `low`, `cut` · remote: `pct`, `via` (`read` / `notify`)
`voice`	`seq`, `codec`, `bytes` (hex) — one notification
`voice_ctl`, `voice_state`	ATV control (`op`: caps / start / stop / search / mic_error) · stream active
`notify`	notification from a characteristic with no known role
`hid_data`	a HID input report with **no key usages** (vendor data, e.g. motion): `handle`, `rid`, `bytes`. The bridge decodes LG Magic Remote keys from it into `key` events (`page: "lg"`, `decoder: "lg_magic"`)
`ir`	`proto`, `addr`, `code`, `repeat`, `raw`
`ir_tx_done`	`ok` (`err` when refused, e.g. battery low)
`log`	`level`, `msg`

Bridge-only: `{"ev":"bridge","connected":…,"fw":…,"port":…,"reason":"nordic-sniffer-image","hint":…}` and `{"ev":"session_log","on":…,"name":…,"url":…,"lines":…}`. Bridge-only command (not sent to the firmware): `{"cmd":"session_log","on":true|false}` — start / stop the persistent log; `{"cmd":"note","text":"…"}` — write a marker line into the session log (the test runner uses it).

Reason and error codes

Firmware messages carry numeric codes. The bridge looks each one up (`backend/reason_codes.py`) and shows its meaning right after the code, everywhere: Connect ▸ Log, toasts, the session-log file and downloaded logs. For example:

```
disconnected (reason 0x08 = Connection Timeout: link lost — remote out of range, powered off, reset or batteries low)
```

The event also carries a `codes` list (`kind`, `code`, `name`, `hint`) for scripts. Look a code up from a terminal with `.venv/bin/python -m backend.reason_codes hci 0x3e` (kinds: `hci`, `smp`, `gatt`, `nrf`).

Message wording	Code family
`disconnected (reason 0x..)`	HCI error / disconnect reason
`pairing failed: 0x..`	SMP pairing failure
`… (status 0x....)`	GATT/ATT status of a read or write
`… err 0x…`, `… failed: 0x…`	nRF SoftDevice API error

HCI error / disconnect reason (`hci`)

Code	Meaning	What it usually means on the jig
`0x00`	Success	
`0x01`	Unknown HCI Command	
`0x02`	Unknown Connection Identifier	the link was already gone
`0x03`	Hardware Failure	
`0x04`	Page Timeout	
`0x05`	Authentication Failure	pairing / encryption was rejected
`0x06`	PIN or Key Missing	the remote no longer has the bond — pair it again (pairing mode)
`0x07`	Memory Capacity Exceeded	
`0x08`	Connection Timeout	link lost — remote out of range, powered off, reset or batteries low
`0x09`	Connection Limit Exceeded	
`0x0A`	Synchronous Connection Limit Exceeded	
`0x0B`	Connection Already Exists	
`0x0C`	Command Disallowed	
`0x0D`	Connection Rejected: Limited Resources	
`0x0E`	Connection Rejected: Security Reasons	
`0x0F`	Connection Rejected: Unacceptable Address	
`0x10`	Connection Accept Timeout Exceeded	
`0x11`	Unsupported Feature or Parameter Value	
`0x12`	Invalid HCI Command Parameters	

Code	Meaning	What it usually means on the jig
`0x13`	Remote User Terminated Connection	the remote closed the link — idle power-save, pairing mode, or it rejected the jig
`0x14`	Remote Terminated: Low Resources	
`0x15`	Remote Terminated: Power Off	the remote is switching off
`0x16`	Connection Terminated by Local Host	the jig closed the link (Disconnect, or a silent link was dropped)
`0x17`	Repeated Attempts	
`0x18`	Pairing Not Allowed	the remote refuses to pair — put it in pairing mode
`0x19`	Unknown LMP PDU	
`0x1A`	Unsupported Remote Feature	
`0x1B`	SCO Offset Rejected	
`0x1C`	SCO Interval Rejected	
`0x1D`	SCO Air Mode Rejected	
`0x1E`	Invalid LL Parameters	
`0x1F`	Unspecified Error	
`0x20`	Unsupported LL Parameter Value	
`0x21`	Role Change Not Allowed	
`0x22`	LL Response Timeout	the remote stopped answering a link-layer procedure
`0x23`	LL Procedure Collision	
`0x24`	LMP PDU Not Allowed	
`0x25`	Encryption Mode Not Acceptable	
`0x26`	Link Key Cannot Be Changed	
`0x27`	Requested QoS Not Supported	
`0x28`	Instant Passed	a connection update came too late — usually radio interference
`0x29`	Pairing With Unit Key Not Supported	
`0x2A`	Different Transaction Collision	
`0x2C`	QoS Unacceptable Parameter	
`0x2D`	QoS Rejected	
`0x2E`	Channel Classification Not Supported	
`0x2F`	Insufficient Security	
`0x30`	Parameter Out of Mandatory Range	
`0x32`	Role Switch Pending	
`0x34`	Reserved Slot Violation	
`0x35`	Role Switch Failed	
`0x36`	Extended Inquiry Response Too Large	

Code	Meaning	What it usually means on the jig
`0x37`	Secure Simple Pairing Not Supported by Host	
`0x38`	Host Busy - Pairing	
`0x39`	Connection Rejected: No Suitable Channel	
`0x3A`	Controller Busy	
`0x3B`	Unacceptable Connection Parameters	
`0x3C`	Advertising Timeout	
`0x3D`	Connection Terminated: MIC Failure	encryption keys don't match — the bond is stale; pair again
`0x3E`	Connection Failed to Be Established	the remote never answered — not in pairing mode, or it only accepts its own host (NOT PAIRING)
`0x3F`	MAC Connection Failed	
`0x40`	Coarse Clock Adjustment Rejected	
`0x41`	Type0 Submap Not Defined	
`0x42`	Unknown Advertising Identifier	
`0x43`	Limit Reached	
`0x44`	Operation Cancelled by Host	
`0x45`	Packet Too Long	

SMP pairing failure (`smp`)

Code	Meaning	What it usually means on the jig
`0x00`	Success	
`0x01`	Timeout	pairing took more than 30 s — the remote stopped responding
`0x02`	Invalid PDU	
`0x81`	Passkey Entry Failed	
`0x82`	OOB Not Available	
`0x83`	Authentication Requirements Not Met	the remote wants a stronger pairing method than Just Works
`0x84`	Confirm Value Failed	
`0x85`	Pairing Not Supported	the remote refuses to pair now — put it in pairing mode
`0x86`	Encryption Key Size Too Short	
`0x87`	SMP Command Not Supported	
`0x88`	Unspecified Reason	often: the remote is not in pairing mode
`0x89`	Repeated Attempts	too many tries — wait a few seconds
`0x8A`	Invalid Parameters	
`0x8B`	DHKey Check Failed	

Code	Meaning	What it usually means on the jig
`0x8C`	Numeric Comparison Failed	
`0x8D`	BR/EDR Pairing in Progress	
`0x8E`	Cross-transport Key Derivation Not Allowed	

GATT/ATT status (`gatt`)

Code	Meaning	What it usually means on the jig
`0x0000`	Success	
`0x0001`	Unknown or not applicable	
`0x0100`	ATT: Invalid Error Code	
`0x0101`	ATT: Invalid Handle	
`0x0102`	ATT: Read Not Permitted	
`0x0103`	ATT: Write Not Permitted	
`0x0104`	ATT: Invalid PDU	
`0x0105`	ATT: Insufficient Authentication	needs a bonded link — press Bond first
`0x0106`	ATT: Request Not Supported	
`0x0107`	ATT: Invalid Offset	
`0x0108`	ATT: Insufficient Authorization	
`0x0109`	ATT: Prepare Queue Full	
`0x010A`	ATT: Attribute Not Found	
`0x010B`	ATT: Attribute Not Long	
`0x010C`	ATT: Insufficient Encryption Key Size	
`0x010D`	ATT: Invalid Attribute Value Length	
`0x010E`	ATT: Unlikely Error	
`0x010F`	ATT: Insufficient Encryption	the link is not encrypted yet — press Bond first
`0x0110`	ATT: Unsupported Group Type	
`0x0111`	ATT: Insufficient Resources	
`0x0112`	ATT: Database Out of Sync	
`0x0113`	ATT: Value Not Allowed	
`0x01FC`	ATT: Write Request Rejected	
`0x01FD`	ATT: CCCD Improperly Configured	
`0x01FE`	ATT: Procedure Already in Progress	
`0x01FF`	ATT: Out of Range	
`0xFFFF`	No response	the link dropped or the remote didn’t answer before the operation finished (jig)

nRF SoftDevice error (`nrf`)

Code	Meaning	What it usually means on the jig
`0x0000`	NRF_SUCCESS	
`0x0001`	NRF_ERROR_SVC_HANDLER_MISSING	
`0x0002`	NRF_ERROR_SOFTDEVICE_NOT_ENABLED	
`0x0003`	NRF_ERROR_INTERNAL	
`0x0004`	NRF_ERROR_NO_MEM	
`0x0005`	NRF_ERROR_NOT_FOUND	
`0x0006`	NRF_ERROR_NOT_SUPPORTED	
`0x0007`	NRF_ERROR_INVALID_PARAM	
`0x0008`	NRF_ERROR_INVALID_STATE	not possible right now (e.g. already connecting, or not connected)
`0x0009`	NRF_ERROR_INVALID_LENGTH	
`0x000A`	NRF_ERROR_INVALID_FLAGS	
`0x000B`	NRF_ERROR_INVALID_DATA	
`0x000C`	NRF_ERROR_DATA_SIZE	
`0x000D`	NRF_ERROR_TIMEOUT	
`0x000E`	NRF_ERROR_NULL	
`0x000F`	NRF_ERROR_FORBIDDEN	
`0x0010`	NRF_ERROR_INVALID_ADDR	
`0x0011`	NRF_ERROR_BUSY	the radio stack is busy with another operation — retry
`0x0012`	NRF_ERROR_CONN_COUNT	
`0x0013`	NRF_ERROR_RESOURCES	
`0x3001`	BLE_ERROR_NOT_ENABLED	
`0x3002`	BLE_ERROR_INVALID_CONN_HANDLE	the link was already gone when the jig tried to use it
`0x3003`	BLE_ERROR_INVALID_ATTR_HANDLE	
`0x3004`	BLE_ERROR_INVALID_ADV_HANDLE	
`0x3005`	BLE_ERROR_INVALID_ROLE	
`0x3006`	BLE_ERROR_BLOCKED_BY_OTHER_LINKS	
`0x3100`	BLE_ERROR_L2CAP_CID_IN_USE	
`0x3200`	BLE_ERROR_GAP_UUID_LIST_MISMATCH	
`0x3201`	BLE_ERROR_GAP_DISCOVERABLE_WITH_WHITELIST	
`0x3202`	BLE_ERROR_GAP_INVALID_BLE_ADDR	
`0x3203`	BLE_ERROR_GAP_WHITELIST_IN_USE	
`0x3204`	BLE_ERROR_GAP_DEVICE_IDENTITIES_IN_USE	
`0x3205`	BLE_ERROR_GAP_DEVICE_IDENTITIES_DUPLICATE	

Code	Meaning	What it usually means on the jig
`0x3300`	BLE_ERROR_GATTC_PROC_NOT_PERMITTED	another GATT operation is still running
`0x3400`	BLE_ERROR_GATTS_INVALID_ATTR_TYPE	
`0x3401`	BLE_ERROR_GATTS_SYS_ATTR_MISSING	

Pins

Function	XIAO	nRF52840
IR receive (Unit IR white)	D0	P0.02
IR transmit, 38 kHz (Unit IR yellow)	D1	P0.03
Battery sense	—	P0.31 via 1 MΩ / 510 kΩ; P0.14 held LOW
Blue LED	—	P0.06 (on while a remote is connected)

Files

Path	Purpose
`backend/bridge.py`	entry point: serial ⇄ WebSocket bridge and web server
`backend/tool_proto.py`	firmware protocol: encode/parse, serial link, probe
`backend/flash.py`	firmware switching via the UF2 bootloader
`backend/sim_device.py`	simulated firmware for `--simulate`
`backend/hid_usage.py`	HID usage → key names (source of the firmware's `usage_names.h`)
`backend/adpcm.py`	IMA-ADPCM → WAV
`backend/report.py`	test report model, JSON + HTML
`backend/session_log.py`	persistent session log (`reports/logs/`)
`backend/lg_magic.py`	LG Magic Remote key decoder (IR command codes inside the motion stream)
`backend/reason_codes.py`	lookup tables for HCI / SMP / GATT / nRF codes in firmware messages
`backend/decode.py`	BLE packet decoders for `ble_probe.py`
`webapp/`	the web app; `js/gatt_info.js` = GATT names, decoders, report-map parser
`firmware/xiao_test_tool/`	firmware source (+ its README)
`firmware/build.sh`, `make_uf2.py`	build and UF2 packaging
`hardware/`	enclosure STLs, parametric `bt_case.scad`, IR wiring SVG
`extcap/`	Nordic Wireshark plugin incl. `SnifferAPI/`
`doc/`	this manual; `doc/src/` regenerates its figures and screenshots
`reports/`	exported reports and voice captures

Glossary

Term	Meaning
BLE	Bluetooth Low Energy
Bond	stored pairing keys that let two devices reconnect encrypted without pairing again
Central / peripheral	the host (TV, jig) / the device (remote)
CCCD	descriptor the host writes to turn notifications on
DUT	device under test
GATT	the remote’s table of services and characteristics
HID	Human Interface Device — keyboards, remotes; key presses are HID reports
IRK / RPA	identity key / resolvable private address derived from it
LESC	LE Secure Connections pairing (keys never cross the air)
PWA	progressive web app — the web app installed on a phone
STB	set-top box
UF2	the drag-and-drop firmware file format of the XIAO bootloader